MADGuard: A High-Performance Microservice Anomaly Detection System With Multidimensional Data Fusion and Temporal Causal Analysis

Yanshang Yin, Tiantian Zhu, *Member, IEEE*, Tieming Chen, and Mingqi Lv

***Abstract*—With the widespread adoption of microservice architectures, the security threats they face have become increasingly sophisticated. Existing anomaly detection methods based on system calls exhibit significant limitations in three key aspects: multidimensional data fusion, temporal causality modeling, and forensic analysis of anomalies. This paper proposes MADGuard, a provenance graph-based anomaly detection system for microservices. MADGuard addresses these challenges through three key innovations: (1) It constructs a native provenance graph by integrating multisource services and multidimensional data, employing feature hashing and positional encoding for efficient graph representation; (2) The system introduces a Temporal Graph Network (TGN) model combined with edge reconstruction error and Inverse Document Frequency (IDF) weighting, achieving a 15. 07% improvement in the F1 score compared to existing methods; (3) For the first time in microservice security, an integrated forensic analysis module is implemented, allowing rapid anomaly path reconstruction through aggregated anomaly subgraphs. Comprehensive evaluations on typical microservice benchmarks (TeaStore, RobotShop, SockShop) demonstrate MADGuard's superior performance: 94.08% detection accuracy, significantly outperforming state-of-the-art approaches while maintaining practical operational efficiency.**

***Index Terms*—Microservices security, provenance graph, temporal graph network, anomaly forensics, anomaly detection.**

I. Introduction

THE MICROSERVICE architecture improves the scalability and agility of the system by decoupling monolithic applications into lightweight, loosely coupled services [1], [2], [3]. This architecture is characterized by service modularization, independent deployment, decentralized governance, and API-based communication mechanisms [4], [5]. These characteristics enable each microservice to be independently scalable and updatable, but also introduce unique security challenges: (1) Dynamic Scaling: The dynamic scaling mechanism causes continuous changes in service instance topology, leading to an exponential increase in the attack surface [6]. (2) Cross-Network Communication: Frequent communication between services renders traditional perimeter defense models ineffective. Middleware components, such as API gateways and service meshes, become new targets for attacks [6]. (3) Containerized Deployment: The use of containerization introduces cloud-native security risks, including image vulnerabilities and container escape [7], [8], [9]. Once a single microservice is compromised, attackers can quickly move laterally to penetrate the entire system, potentially causing catastrophic consequences such as data breaches and service interruptions [10]. These security challenges mean that anomaly detection systems in microservices environments must not only address traditional network attacks but also meet new defense needs, such as distributed tracing and zero-trust verification [11], [12]. As a result, anomaly detection has become a cornerstone for ensuring the stable operation of cloud-native systems [13].

Current research in microservice anomaly detection focuses primarily on analysis of system calls using machine learning or signature-based methods [14], [15], [16], [17], [18], [19]. However, three critical limitations persist in enterprise environments:

(1) Multi-dimensional data fusion: Existing approaches such as CDL [20] oversimplify detection by relying solely on system call frequencies, neglecting crucial contextual features (process hierarchies, file process patterns, network flows). This feature sparsity severely limits the detection accuracy for sophisticated attacks.

(2) Shallow temporal causality: Methods such as STIDE-BoSC [21], CHIDS [22], and ReplicaWatcher [23] analyze short system call sequences without modeling temporal dependencies, leading to high false negatives for multistage attacks. For example, our TeaStore case study (Section II) reveals how these methods fail to detect low-frequency attack patterns masked within normal operational sequences.

(3) Lack of anomaly forensic analysis: Current systems [20], [21], [22], [23] lack causal analysis capabilities, providing isolated alerts without reconstruction of the attack path. This gap significantly impedes incident response, particularly for cross-service attacks requiring end-to-end visibility.

Table I presents the detection capabilities of existing microservice anomaly detection systems across various dimensions. However, no existing method can simultaneously

Received 19 March 2025; revised 6 September 2025; accepted 15 November 2025. Date of publication 24 November 2025; date of current version 29 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2028, Grant 62002324, and Grant 62372410; in part by the Fundamental Research Funds for the Provincial Universities of Zhejiang under Grant RF-A2023009; in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LQ21F020016, Grant LD22F020002, and Grant LZ23F020011; and in part by "Pioneer" and "Leading Goose" R&D Program of Zhejiang under Grant 2025C01082 and Grant 2025C01013. The associate editor coordinating the review of this article and approving it for publication was R. Riggio. *(Corresponding authors: Tiantian Zhu; Tieming Chen.)*

The authors are with the School of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: ttzhu@zjut.edu.cn; tmchen@zjut.edu.cn).

Digital Object Identifier 10.1109/TNSM.2025.3634590

TABLE I
THE EXISTING ANOMALY DETECTION METHODS ARE COMPARED IN THREE DIMENSIONS: DATA FEATURE MULTI-DIMENSIONAL FUSION, TIME SERIES CAUSAL CORRELATION, AND ANOMALY FORENSICS ANALYSIS

Existing Anomaly Detection Methods	Multi-dimensional data	Temporal causality	Anomaly forensics analysis
MADGuard	●	●	●
STIDE-BoSC [21]	○	○	○
CHIDS [22]	○	○	○
CDL [20]	●	○	○
ReplicaWatcher [23]	●	○	○

Note: ●indicates support for the feature; ○indicates no support for the feature.

address the three main challenges. The provenance graph represents the control flow and data flow relationships between system subjects (such as processes, threads, etc.) and objects (such as files, network sockets, etc.) through a directed graph, thereby reflecting the causal relationship between system events. Due to their rich semantics and robustness, provenance graphs are widely used in network intrusion detection [24], [25], [26]. To address these challenges and improve the performance of anomaly detection systems in microservice environments, this paper designs a microservice anomaly detection system based on provenance graphs. The system integrates multidimensional information from the microservice environment and combines graph learning to construct an effective anomaly detection model.

Considering the distributed nature of the microservice architecture, this paper collects key information events covering multiple dimensions (e.g., processes, files, networks) in a multisource service environment. To integrate these multi-source and multi-dimensional data information without causing confusion, this paper employs service identification (mid) and Hasche algorithm to uniquely identify system objects across different services. Based on these objects, related events are merged. After data fusion, these complex data are uniformly characterized to facilitate subsequent model learning. Inspired by previous work [27], this paper adopts feature hashing and location encoding techniques to efficiently encode multi-source information in the microservice environment into compact and semantic-rich feature vectors. Furthermore, a time-sliding window mechanism with a time interval T is designed to accumulate feature vectors in real-time. This mechanism not only maintains data timeliness but also addresses the issue of insufficient multi-dimensional fusion in existing data features. Experimental results in the deployed microservice environment demonstrate that embedding feature vectors through the time-sliding window mechanism effectively improves anomaly detection accuracy and reduces the false alarm rate.

To address the issue that existing anomaly detection methods do not fully consider temporal causal relationships in data sequences, this paper uses the structural characteristics of provenance graphs to construct data information. By structuring data into provenance graphs, temporal continuity and causality are strengthened. The Temporal Graph Network (TGN) model is also introduced to dynamically capture complex temporal causal relationships between data [28]. The unique advantage of the TGN model is its ability to analyze both temporal and spatial characteristics of graph sequences, enabling the identification of local and global abnormal events with high accuracy. This capability significantly enhances the accuracy and efficiency of anomaly detection. Moreover, by adopting provenance graphs, the original data's temporal information is retained, and the causal relationships between data are more intuitively represented, providing stronger explanatory and predictive power in complex data environments.

Abnormal forensic analysis can infer the chain of abnormal events and restore the abnormal path based on the provenance graph. However, as time passes and the volume of data increases, the size of the provenance graphs increases exponentially [29], making manual construction impractical. Existing forensic methods in other fields construct concise summary graphs to restore abnormal event paths, reducing the need for human intervention [30]. However, in microservice environments, virtualization characteristics can confuse identical data information across different services, leading to inaccurate exception source localization. To address this, this paper introduces unique identifiers to distinguish services and designs an anomaly forensic analysis algorithm. The experimental results show that the proposed algorithm significantly reduces the workload of security staff, clarifies abnormal data in different microservice environments, and shortens the forensic analysis time. Furthermore, this paper is the first to integrate an anomaly forensic analysis module into the microservice anomaly detection system, enhancing overall system efficiency and accuracy.

To verify the effectiveness of the proposed method, experiments were conducted in three representative microservice environments, simulating three typical attack scenarios. The results indicate that the proposed anomaly detection system outperforms existing methods, with a 15.07% improvement in detection accuracy. The main contributions of this paper are:

(1) Innovative System Design: A novel microservice anomaly detection system based on provenance graphs is proposed. By integrating multi-source services and multi-dimensional data, the system constructs a provenance graph in the microservice environment and leverages graph learning to significantly enhance anomaly detection accuracy.

(2) Enhanced Data Features: Advanced feature hashing techniques are employed to efficiently fuse multi-source, multi-dimensional data, optimizing data samples. The time-sliding window mechanism enriches data diversity and timeliness, providing more comprehensive and accurate data for anomaly detection.

(3) Integrated Forensic Analysis: The system introduces an anomaly forensic analysis module for the first time, enabling accurate restoration of abnormal events and saving considerable time in security forensic analysis.

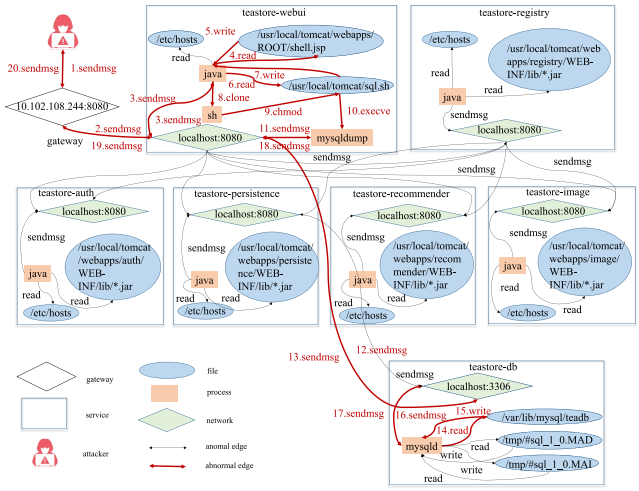

Fig. 1. Teastore-webshell attack. Initially, the attacker leveraged the WebUI service to upload a WebShell file named *shell.jsp*, thereby gaining unauthorized access to the server. Subsequently, through local port scanning, the attacker obtained the database connection details. Using this information, they created a Shell script named *sql.sh* to access the remote user database. Finally, the attacker employed the *mysqldump* tool to export sensitive information, including user login credentials and credit card details.

(4) Comprehensive System Evaluation: The system's performance is comprehensively evaluated in three real-world microservice environments, demonstrating excellent performance in key metrics such as detection accuracy, detection time, and overhead.

II. Motivation

This chapter presents a critical examination of webshell attack detection in microservice environments through a case study on the TeaStore platform, highlighting both the constraints of conventional anomaly detection techniques and the advantages of our proposed methodology.

The TeaStore benchmark—a distributed system encompassing authentication, procurement, and three other independently scalable services—serves as the experimental foundation. As depicted in Figure 1, we simulate a sophisticated attack chain initiated by exploiting CVE-2017-12625, an arbitrary file write vulnerability in the WebUI component. Attackers first intercept service communications via Burpsuite, injecting a malicious JSP file into the Tomcat directory to establish remote code execution. Subsequent abuse of database misconfigurations [31] enables credential harvesting and deployment of *SQL.sh* scripts, ultimately compromising sensitive customer financial data through orchestrated database queries.

For the aforementioned attack scenario, existing anomaly detection methods such as ReplicaWatcher and CHIDS detect window-based anomaly events by identifying isolated features. For example, they may detect the */usr/local/Tomcat/webapps/ROOT* folder path in the file direction feature or the *shell.jsp* file and Java process in the process feature (Figure 2). However, these methods analyze features in isolation, leading to a high false-negative rate. Even if malicious processes or files are identified, it is challenging to trace the source of the anomaly.


```
{
  "list_distinct_commands": [],
  "list_distinct_cwds": ["/", "/usr/local/tomcat/"],
  "list_distinct_args": [
    "-c java -jar /usr/local/tomcat/bin/
        dockermemoryconfigurator.jar ${
        TOMCAT_HEAP_MEM_PERCENTAGE}; /usr/local/tomcat/"
  ],
  "list_distinct_exe": [
    "/pause",
    "/usr/local/java/jdk-11.0.0.0.2/bin/java",
    "/bin/sh"
  ],
  "pod_id": "teastore-webui-585849d98b-7rrmg",
  "begin_time": "11:04:42:01432827",
  "end_time": "11:05:12.837896207",
  "list_distinct_syscalls": [
    "lseek", "write", "sched_getaffinity", "accept",
    "fcntl", "getsockname", "exit", "getrusage"
  ],
  "list_distinct_syscall_category": [
    "memory", "scheduler", "file", "net",
    "IPC", "ipc", "other", "wait", "internal", "process"
  ],
  "list_distinct_directories": [
    "/usr/local/tomcat/conf/Catalina/localhost",
    "/usr/local/tomcat/webapps/tools.descartes.teastore.
        webui/WEB-INF/lib",
    "/etc",
    "/usr/local/tomcat/webapps/ROOT",
    "/usr/local/tomcat/webapps"
  ],
  "list_distinct_filenames": [ "opentracing-noop-0.31.0.jar
      ", "shell.jsp", "aopalliance-repackaged-2.6.1.jar", "
      yum.conf", "shadow" ],
  "list_distinct_file_operations": ["directory", "ipv4", "
      file", "pipe"],
  "list_distinct_procs": [
    "C1", "Sweeper", "Common-Cleaner", "process",
    "pause", "http-nio-8080-A", "sh", "ContainerBackgr", "
        java"
  ]
}
```


Fig. 2. ReplicaWatcher and CHIDS anomaly detection methods identify feature.

CDL anomaly detection method detects abnormal events by analyzing the frequency of system calls. Specifically, CDL counts the frequency of system call events every second and identifies deviations from the baseline based on this frequency (Table II). However, this method is only effective for high-frequency attacks and is limited in detecting low-frequency attacks. For instance, in the case of the CVE-2017-12615 vulnerability attack, the attacker's impact on system calls is minimal, and the normal and abnormal system call frequency characteristics are nearly indistinguishable, making it difficult to detect the attack.

STIDE-BoSC method combines the STIDE [32] and Bosc [33] techniques to detect abnormal events. It counts the frequency of different system calls within the current time window based on their temporal sequence and maintains a system call vector database with a window size of 10. During detection, the system call vector of an epoch is matched against the database. If the mismatched vector exceeds a certain threshold, the epoch is deemed abnormal. Although this method considers the order of system calls, even minor changes in the system can trigger false alarms, resulting in a high false-positive rate.

III. Threat Model

To construct a classical threat model, we focus on microservices deployed in Kubernetes environments [34], as most

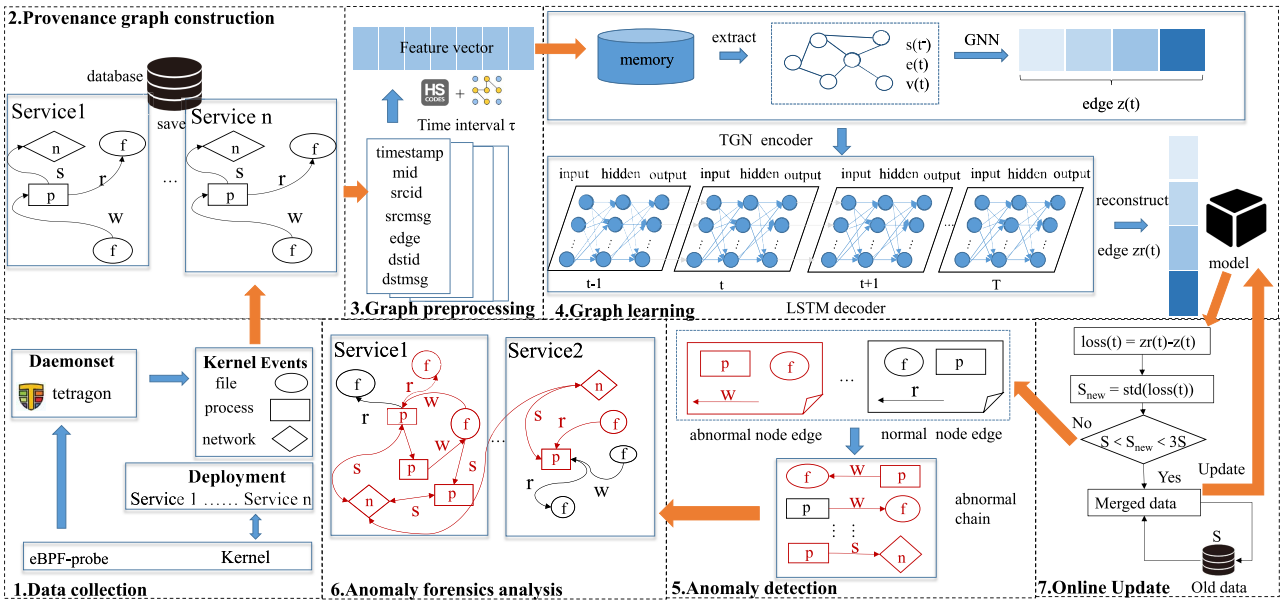

Fig. 3. MADGuard system architecture.

TABLE II
THE NORMAL AND ABNORMAL SYSTEM CALL FREQUENCY CHARACTERISTICS OF CDL STATISTICS

Timestamp	System Call Frequency				
	switch	stat	lstat	close	read
1735521966740295269	32	258	62	0	192
1735521966840295269	8	0	0	0	0
1735521966940295269	9	0	0	0	0
1735521967040295269	7	2	0	0	0
1735521967140295269	21	2	0	0	0
1735521967240295269	9	0	0	0	0
1735521967340295269	29	0	0	0	0
1735521967440295269	10	0	0	0	0
1735521967540295269	9	0	0	0	0
1735521967640295269	25	88	62	64	0
1735521967740295269	9	0	0	0	0
1735521967840295269	7	0	0	0	0
1735521967940295269	47	104	104	0	64
1735521968040295269	8	0	0	0	0
1735521968140295269	19	0	0	0	0
1735521968240295269	54	62	0	0	48

Note: red area indicates abnormal data.

existing microservices are hosted there. Similarly to previous studies [20], [21], [22], [23], this article examines attacks initiated by external adversaries that exploit open vulnerabilities. Specifically, attackers target microservices through software-level vulnerabilities, including remote code execution, directory traversal, and code injection attacks. We assume that attackers leave identifiable traces after successfully compromising a microservice. Additionally, we assume that the collected system log data is complete, accurate, and untampered. This paper does not address internal attacks, system-level attacks, side-channel attacks, or hardware-related attack scenarios.

IV. DESIGN OF ANOMALY DETECTION SYSTEM

This paper proposes a microservice anomaly detection system addressing four critical dimensions: data diversity, temporal sequencing, causality, and forensic analysis. As illustrated in Figure 3, the architecture comprises five core components: provenance graph construction, graph preprocessing, graph learning, anomaly detection, and forensic analysis. The operational workflow proceeds as follows: (1) The system collects process, file, network log data from multi-source services. Temporal and causal relationships are analyzed to construct comprehensive provenance graphs across distributed services. (2) Hash encoding and positional encoding techniques are used to preprocess the node and edge information in the provenance graph into feature vectors. For each 1-hop event, composite feature vectors are generated by concatenating node-edge attributes. Dynamic time windows τ to generate real-time embedding feature vectors. (3) Considering the temporal dynamic characteristics of data, the system constructs a TGN dynamic time series network to learn the temporal and spatial characteristics of benign log data. (4) An anomaly detector computes edge reconstruction errors per window. Threshold-exceeding errors trigger alerts and forensic pipeline activation. (5) The forensic module reconstructs anomaly propagation paths through subgraph analysis and visualization.

A. Provenance Graph Construction

The construction of traditional provenance graph is mainly aimed at a single host environment, while microservices run in a distributed manner. Therefore, information from multiple service sources must be integrated when building provenance diagrams in a microservice environment. (1) Multi-source Data Integration: Leveraging Kubernetes deployment characteristics, we collect pod-specific logs from cluster nodes, including process execution traces, file operations, and network activities. Hash-based entity resolution reconciles system objects across services while maintaining pod isolation through unique service identifiers (mid). (2) Definition and Representation of Provenance Graph:

TABLE III
ATTRIBUTES AND FEATURES

	Attribute	Feature
Node	process	proc, args
	file	filename, directory
	network	srcip, srcport, dstip, dstport
Edge	syscall	type (read,write,close,connect,sendmsg)

According to the definition of the system-level provenance graph [24], the system call event E can be represented by a < subject, object, time, operation >quadruple. Subject and object represent application process, file and network information, and operation represents system call types, such as read, write, etc. The entire provenance graph G is represented by (S, O, E) and includes all subjects S, objects O, and events E. Given the distributed nature of microservices, podname is introduced when building provenance graphs to distinguish between different pod events. Microservice system call event E_m by $<mid, subject, object, time, operation>$. To represent the provenance diagram G_m by (ID, S, O, E) to show that. (3) Data Storage Optimization: Database schemas organize subject/object nodes and operational edges for efficient retrieval, reducing preprocessing latency.

B. Graph Preprocessing

Before graph learning, it is necessary to preprocess the provenance graph so that the model can learn the information in the system log.

(1) Encoding Node, Edge, and Position Information. To efficiently encode edge, node, and node position information within the graph, this section integrates Hash encoding and node position encoding techniques. These techniques transform the nodes, edges, context information, and node positions in the provenance graph into compact feature vectors. Traditional encoding methods, such as one-hot and BoSC, can convert features into sparse vectors, but fail to capture rich contextual semantics. In contrast, Hash encoding maps high-dimensional feature information to low-dimensional vectors through feature hashing, albeit without capturing node position information. To address this limitation, we combine position encoding technology to extract node features from the provenance graph. Additionally, to capture the complexity of system behavior, we select nine features closely related to system behavior, including process name, file path, network source-destination IP and port, and system call type. These features can effectively describe system behavior, as detailed in Table III.

Using Hash encoding, we convert node, edge, and node context information strings into fixed-length feature vectors. This is achieved through two Hash functions: one maps element n_j to the *i*-th dimension of the feature vector, while the other maps n_j to ± 1. The formula is defined as follows:

$$\Phi(n) = \sum_{j:h(s_j)=i} \xi(n_j) \tag{1}$$

(2) Restoring Node Position Relationships. Although the above feature encoding restores the relationships between nodes, it does not accurately represent the positional relationships of nodes within the graph. To address this, we employ position encoding to restore the positional relationships.

$$n = \Phi(n) + position(n) \tag{2}$$

(3) Encoding Events in 1-Hop Subgraphs. To effectively apply the encoded features to graph learning, we encode events represented by each 1-hop subgraph. This coding process strictly follows chronological order to ensure the temporality of events, which is essential to reveal the dynamic changes and causality of events. Specifically, we generate a comprehensive event feature vector by stacking the feature vectors of timestamps, source context, edge types, and destination context.

$$G_{m_t} = \sum_{t_{timestart}}^{t_{timeend}} [src_t, dst_t, t, msg_t] \tag{3}$$

where

$$msg_t = [mid_t, edge_t, srcmsg_t, dstmsg_t] \tag{4}$$

(4) Dynamic Window Embedding. Sliding windows τ create temporal embeddings balancing real-time responsiveness and anomaly progression capture.

$$Z_\tau = \sum^{T} G_\tau \tag{5}$$

Graph Learning Given the temporal dynamics of log data and their corresponding provenance graphs, conventional Graph Neural Networks (GNNs) prove inadequate as they primarily handle static graph structures. To address this limitation, we employ a TGN with memory-enhanced update mechanisms to capture spatio-temporal patterns in evolving graphs.

MADGuard selects TGN (Temporal Graph Network) as the core temporal modeling architecture due to its deep adaptation to the inherent dynamism, distribution, and causal complexity of microservice environments. Compared to general temporal graph models (such as TGAT or EvolveGCN), TGN demonstrates significant advantages in the following three aspects:

First, TGN introduces unique microservice identifiers (mid) and Pod names (podname) to represent cross-service events as <mid, subject, object, time, operation> quintuples. This mechanism enables the spatiotemporal correlation modeling of distributed multi-source heterogeneous events, ensuring that interactions between different service nodes can be accurately captured and associated. It overcomes the topological complexity brought by dynamic scaling and cross-node communication in microservice environments.

Second, TGN combines a dynamic time window mechanism (such as a 1-minute sliding window) to maintain the temporal continuity of events while effectively covering the complete anomaly propagation chain (such as file upload, database access, and data export in a WebShell attack). This design balances real-time responsiveness and the integrity of anomaly progression, avoiding feature fragmentation caused by overly short windows (e.g., a significant drop in Recall with a 30-second window) or noise interference introduced by overly long windows (e.g., a decrease in Precision with a 90-second window).

Third, TGN's memory-enhanced mechanism dynamically aggregates historical node states, edge attributes, and current features through an encoder, enabling the maintenance of long-term cross-service causal dependencies. This is particularly suitable for detecting multi-stage low-frequency attacks in microservices. The mechanism performs outstandingly in attack scenarios that require long-term context awareness, such as code injection and information leakage. For related experimental evidence, see Section V-E.

The TGN framework operates through coordinated encoder-decoder components. The encoder is used for feature embedding from the provenance graph, and the decoder predicts the edge type according to the embedded feature vector. Specifically:

Encoder: Dynamically embeds edge features through temporal aggregation. For an edge e_t at timestamp *t*, its embedding $z(t)$ synthesizes three information streams: Historical node states $s(t^-)$ preceding *t*; Edge attributes $e(t)$; Current node features $v(t)$ via graph neural network propagation:

$$z(t) = GNN(s(t), e(t), v(t)) \tag{6}$$

Decoder: Predicts edge types through sequential modeling. A Long Short-Term Memory (LSTM) network decodes the temporal dependencies between nodes states, generating probabilistic edge type predictions:

$$pred(e(t)) = LSTM(z(t)) \tag{7}$$

The output $pred(e(t)) \in \mathbb{R}^{1\times 5}$ represents a probability distribution over five predefined edge types, allowing reconstruction-based anomaly detection through prediction errors.

C. Anomaly Detection

To efficiently identify anomalous events, we segment the entire provenance graph into consecutive, fixed-length time window sequences. Within each time window, we detect suspicious edges and nodes. Based on these suspicious edges and nodes, we construct a suspicious propagation chain. We then calculate the anomaly score of the suspicious propagation chain. If the anomaly score exceeds the benign data threshold θ, we conclude that an anomaly exists within the current window.

Our system is tailored for multi-container microservices, using 1-minute detection interval. This avoids the resource contention of high-frequency detection seen in complex architectures. The 1-minute window also allows for detailed data analysis with Tetragon, covering process behaviors and network traffic. Unlike methods like ReplicaWatcher with a 30-second interval, our approach captures the full propagation chain of progressive anomalies like resource leaks. Combined with a four-day pre-trained model, it balances real-time response with long-term pattern generalization, optimizing resource use, analysis depth, and detection accuracy.

(1) **Suspicious edge identification**: We utilize a well-trained and high-performance anomaly detection model to reconstruct edges within each time window. Meanwhile, we calculate the reconstruction error of these edges. If the reconstruction error is notably high, it signals that the current subgraph strays from the expected pattern, hinting at potential suspicious activities.

(2) **Suspicious node identification**: Suspicious edges often link to suspicious nodes. After identifying an suspicious edge, we check connected nodes for unusual behavior. Furthermore, in real-world environments, attackers might use common processes for sensitive operations, like accessing sensitive files */etc/passwd*. In such cases, uncommon nodes may emerge, exhibiting suspicious behaviors. We use Inverse Document Frequency (IDF) [35] to spot these sensitive files. The mathematical formula is as follows:

$$IDF_v = log\left(\frac{N}{df_v + 1}\right) \tag{8}$$

where N is the number of nodes in the current time window, df_v is node v's occurrence frequency in the current time window. If IDF_v exceeds the threshold, the node is flagged.

(3) **Anomaly propagation chain construction**: Once identifying suspicious edges and nodes within each time window, we trace related events to form a one-hop event queue. We then calculate the anomaly score for the corresponding queue. If all queue scores in the window exceed θ (benign data threshold), we confirm an anomaly. Following this, we construct the anomaly propagation link within the window based on these queues.

D. Anomaly Forensic Analysis

The primary objective of the attack forensics component is to provide security analysts with a streamlined global attack graph, which includes two key tasks: 1) reconstructing the global attack chain; 2) filtering out false positives to refine the attack graph. During the anomaly detection phase, it is common to identify isolated anomalous events without gaining a comprehensive view of the anomalies, which forces security analysts to sift through numerous isolated anomalous nodes to find the source of the attack, a process that is extremely time-consuming and labor-intensive [36]. To address this issue, this paper compresses benign redundant nodes and edges within services based on the one-hop events constructed during the anomaly detection phase, reducing the overhead of graph construction. We have observed that the anomaly scores of anomalous events are significantly higher than those of benign events. Therefore, by ranking the anomaly scores of events and selecting the top-k anomalous events, we can essentially reconstruct the attack path while refining the attack graph.

Furthermore, since existing cross-service trace tracking can only reconstruct the application-level call chains between services and cannot achieve cross-service call chains based on kernel-level logs, external attackers often infiltrate the internal service systems to gain maximum privilege information. Hence, we have designed a global microservices attack graph reconstruction method to track both single-service and cross-service global attack chains. For cybersecurity teams, anomaly forensic analysis is crucial for quickly pinpointing the source of events and restoring affected paths. To expedite this process, we simplify complex anomaly data, including consolidating edges for identical nodes across various times and versions. To accurately trace the source of anomalous events, our module

Algorithm 1 Anomaly Graph Construction

Input: List of anomaly events Q; Anomaly threshold θ; Number of representative events k
Output: Service-Specific anomaly graphs $G_p|p \in pods$; Global anomaly graph G_{global}

1: // Step1: Filter top-k representative anomaly events
2: Initialize $G \leftarrow \{\}$ // maps podnames to lists of events
3: count $\leftarrow 0$
4: **for** each event e in Q **do**
5: compute loss $\leftarrow$ crossEntropy(e)
6: p $\leftarrow$ podname(e)
7: **if** loss $> \theta$ **then**
8: **if** p $\notin$ G **then**
9: G[p] $\leftarrow$ empty list
10: **end if**
11: append e to G[p]
12: count $\leftarrow$ count+1
13: **if** count $\geq$ k **then**
14: break
15: **end if**
16: **end if**
17: **end for**
18: // Step2: Construct anomaly graph for each service
19: initialize $pod_{node} \leftarrow$ empty dictionary // maps nodes to podnames
20: initialize $src_{node} \leftarrow$ empty dictionary // maps podnames to source nodes
21: **for** each podname p $\in$ G **do**
22: $G_p \leftarrow$ create new directed graph
23: **for** each event e $\in$ G[p] **do**
24: **for** each node n in nodes(e) **do**
25: **if** n $\notin pod_{node}$ **then**
26: $pod_{node}[n] \leftarrow$ p
27: **end if**
28: add n to G_p
29: **end for**
30: **for** each edge(u,v) in edges(e) **do**
31: add(u,v) to G_p
32: **end for**
33: src $\leftarrow$ identifySourceNode(e)
34: **if** src $\notin src_{node}[p]$ **then**
35: $src_{node}[p] \leftarrow$ src
36: **end if**
37: **end for**
38: **end for**
39: // Step3: Construct global anomaly graph
40: $G_{global} \leftarrow$ create new directed graph
41: **for** each podname p $\in$ G **do**
42: // add cross-pod edges
43: **for** each node n in $G_p.nodes$ **do**
44: **if** $pod_{node}[n] \neq p$ **then**
45: add edge ($src_{node}[p]$,n) to G_{global} // assuming $src_{node}[p]$ is source node of pod p
46: **end if**
47: **end for**
48: **end for**
49: **return** G_p, G_{global}

compiles anomaly subgraphs from each microservice into a unified global graph. Using this approach, we have developed an algorithm (Algorithm 1) that identifies the top K most significant anomalies by evaluating anomaly values across different service subgraphs. Finally, we employ Graphviz to visualize the complete anomaly path. This tool offers a detailed and interactive representation, aiding in efficient root cause analysis.

In Algorithm 1, the detection of cross-service edges relies on the Pod identification native binding mechanism introduced during the provenance graph construction phase (Section IV-A). This ensures that each system entity (processes, files, network connections) is explicitly associated with its respective Pod. Specifically, we use the eBPF-driven kernel-level monitoring tool Tetragon to collect logs and attach Pod-level metadata (such as pod_id, container paths, and source/destination Pod information for network connections) to each node, rather than relying solely on node names for identification. For example:

Process nodes are uniquely identified by a combination of "process name-Pod name" (e.g., java-teastore-webui, java-teastore-db). Even if the process names are the same, instances in different Pods are treated as separate nodes. File nodes are associated with Pods through their absolute paths within the container (e.g., */usr/local/tomcat/webapps/ROOT/shell.jsp* belongs to the teastore-webui Pod, while */var/lib/mysql/teadb.sql* belongs to the teastore-db Pod). Network nodes are bound to Pods based on the source and destination IP addresses of the connections (e.g., the destination IP *10.0.0.10:3306* belongs to the teastore-db Pod, while the source IP *10.0.0.5* corresponds to the teastore-webui Pod).

During the anomaly graph construction process, Algorithm 1 maintains a podnode dictionary (steps 19–27) to record the Pod IP information of each network node. When processing an anomaly event for a particular Pod, the algorithm checks the network nodes. If it finds that the destination IP of a network node exists in podnode and the corresponding Pod of this destination IP is not the current Pod being processed, then this node is identified as a cross-Pod node (step 43). Subsequently, the algorithm locates the initial attack source node srcnode[p] of the current Pod (obtained by sorting and filtering based on anomaly scores, steps 33–36) and constructs an edge from srcnode[p] to this cross-Pod node n, adding it to the global anomaly graph G_global (step 46).

Taking the TeaStore WebShell attack as an example (Section V-D, Figure 1, Figure 6), when the database process mysqld-teastore-db is accessed by the abnormal process java-teastore-webui (marked as abnormal due to the execution of the malicious file *shell.jsp*) in the teastore-webui Pod, specific sockets are generated. In the teastore-webui Pod, the socket *10.244.2.168:55602→10.102.87.233:3306* is created, where the destination IP *10.102.87.233:3306* points to the database service. In the teastore-db Pod, the socket *10.244.2.167:3306→10.244.2.168:55602* is created. When the algorithm processes the anomaly event for the teastore-webui Pod, it detects that the destination IP *10.102.87.233:3306* corresponds to the teastore-db Pod, not the teastore-webui Pod, and thus identifies this network node as a cross-Pod node. Meanwhile, the algorithm locates the initial attack source node srcnode[teastore-webui] as java-teastore-webui and constructs a cross-service edge from java-teastore-webui to mysqld-teastore-db. This cross-service edge, together with subsequent database operation nodes (such as the node executing mysqldump to export sensitive data), forms a complete attack chain.

It is worth noting that in microservice environments, even if the same node (such as */etc/hosts* or a database IP) is accessed by multiple Pods, Tetragon generates unique socket identifiers for each interaction (e.g., *10.0.0.5:55602→10.0.0.10:3306* vs. *127.0.0.1:3306→127.0.0.1:48921*), and these socket details are preserved through edge attributes (Section IV-B). This

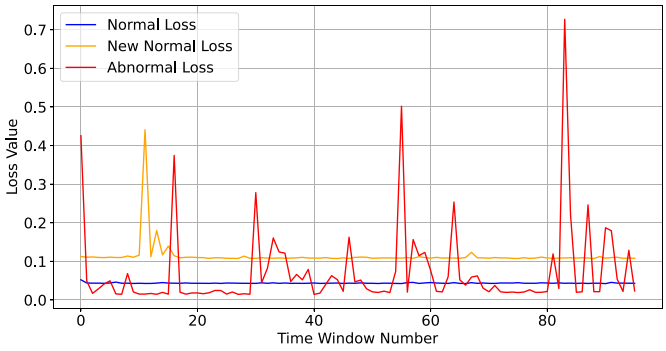

Fig. 4. Loss values comparison.

ensures that the algorithm can accurately distinguish between Pod interactions and avoid missing real cross-service dependencies.

E. Online Update

To address the issue of performance degradation of MADGuard over time, we propose an online update mechanism (Online Update, OU). The core of OU lies in the concept of real-time drift detection and targeted incremental model updates. To automatically identify when model performance degrades due to changes in data distribution (concept drift), OU utilizes the standard deviation of anomaly scores output by the model as a key metric. This approach relies on the analysis of three types of typical data (pure benign daily data, new daily data generated by the evolution of benign behavior, and anomalous daily data). The standard deviation of anomaly scores s for pure benign data is relatively stable; the standard deviation s_{new} produced by the evolution of benign behavior typically falls within 3 times the historical standard deviation s (i.e., $s_{new} \in (s, 3s)$); whereas the standard deviation s_{anom} generated by anomalous data significantly exceeds this range, as shown in Figure 4. Based on this finding, the system maintains a sliding window to record the current baseline of benign behavior and calculates the standard deviation s_{new} of newly incoming data in real-time. When s_{new} falls within the $(s, 3s)$ range, a concept drift alarm is triggered.

Once drift is detected, the system initiates an incremental model update process. To prevent the model from updating solely on new data, which could lead to a "catastrophic forgetting" of old knowledge, the update does not only use the new window data that triggered the drift. The system looks back at an equally sized historical old window of data (representing previously stable normal data). Subsequently, the new window data causing the drift is merged with the retrieved old window data to form a small-scale incremental training dataset. The existing model is then incrementally trained with the merged dataset, allowing the model to learn new patterns while consolidating old knowledge. After successfully updating the model, the reference sliding window data used for detecting drift also needs to be updated, typically refreshing the reference baseline with the new window data that triggered this update, providing the most current standard deviation s baseline for subsequent drift detection.

V. Evaluation

To comprehensively evaluate the effectiveness of our proposed anomaly detection system, we investigate four key research questions through rigorous experimentation:

RQ1. Detection Accuracy: How does our system compare with existing microservice anomaly detection systems in terms of detection performance?

RQ2. Runtime Overhead: What is the operational cost of our anomaly detection system, and how does it compare with state-of-the-art solutions?

RQ3. Forensic Capability: How effectively does the forensic analysis module trace anomaly sources, and what performance impact does it introduce?

RQ4. Component Ablation: What is the individual contribution of each component in MADGuard to the system's performance?

A. Experimental Setup

(1) **Test Environment:** We deployed a Kubernetes cluster with one master node and two worker nodes running Ubuntu 22.04 and Containerd v1.6.33.

Teastore[37], SockShop [38], and RobotShop[38] cover the common multi-technology stacks found in enterprise-level microservices, as shown in Table IV below. These scenarios encompass mainstream languages such as Java, PHP, Node.js, and Go, and their attack surfaces (such as Tomcat remote code execution, PHP code injection, and Node.js path traversal) are consistent with real-world enterprise environments. We have selected Teastore, SockShop, and Teastore as test environments. Regarding log collection tools, since Kubernetes native observability tools (such as service mesh, Istio) focus on application-level call chain monitoring, they are unable to capture kernel-level attack traces (such as access to sensitive files like /etc/passwd, creation of malicious files like shell.jsp). Therefore, MADGuard employs a kernel-level observation tool, Tertagon, which uses eBPF technology to collect process, file, network, and other low-level events, ensuring the complete capture of attack behaviors. The Tetragon monitoring component [39] was deployed as a DaemonSet to ensure non-intrusive data collection across all nodes.

(2) **DataSet**: We injected representative vulnerabilities into three microservice benchmarks (Teastore [37], Sockshop [38], Robotshop [38]) to simulate critical attack scenarios:

WebShell attack (CVE-2017-12615): The attacker first constructs a WebShell file named *shell.jsp* within the webui service to gain unauthorized access to the server. Subsequently, by scanning local ports, the attacker acquires connection details for the teadb database. Using this information, the attacker creates a shell script named *sql.sh* to remotely access the user database. Finally, leveraging the mysqldump utility, the attacker extracts sensitive information, including user login credentials and credit card details.

Code injection (CVE-2024-2961): The attacker creates a malicious *shell.php* file, leverages a remote code execution flaw in the ratings service to run it, and exfiltrates server details (*RobotShopRatingsKernelDevbugContainer.php.meta*) and sensitive file data (*/etc/group*).

TABLE IV
MICROSERVICES PLATFORMS AND THEIR TECHNOLOGIES

Microservice Platform	Service	Language
TeaStore	webui	java
	image	
	authentication	
	recommender	
	persistence	
SockShop	carts	java
	catalogue	mysql
	frontend	node.js
	payment	go
	queuemaster	java
	shipping	java
	user	go
	order	java
	queue	rabbitmq
RobotShop	cart	node.js
	catalogue	node.js
	dispatch	go
	payment	python
	ratings	php
	shipping	java
	user	node.js
	web	nginx

Information leakage (CVE-2017-14849): The attacker exploits a vulnerability in the front-end service to construct malicious file paths (e.g., */../../../../etc/XX*) and access sensitive server files such as *passwd, shells, group, fstab, profile*, and *protocols*.

For each scenario, we collected 4 days of operational data containing more than 30 million normal events and 900,000 anomalous events, as detailed in Tables V and VI.

(3) **Baseline Systems:** We compare against four state-of-the-art approaches:

STIDE-BOSC [21]: It combines STIDE and BOSC techniques to model a sequence of system calls. Specifically, the method uses a time window with a size of 10 to record the frequency of each system call and constructs a system call bag-of-words vector database. In the anomaly detection phase, the new system call bag-of-words vector is compared with the vector in the database. Once the number of mismatched bag-of-words vectors exceeds the set threshold, the current time period is judged to be an anomaly.

CHIDS [22]: This method focuses on identifying unusual sequences of system calls while integrating the sequence's parameters and context information. It divides a long sequence into small segments of fixed length and represents these segments as graphs. By analyzing the degree centrality and weight of the graph, feature vectors such as sequence frequency and parameters are extracted, and these feature vectors are trained by autoencoder. In the anomaly detection stage, by reconstructing these feature vectors, if the reconstruction error exceeds the preset threshold, the sequence is considered to be abnormal.

CDL [20]: This approach focuses on counting the frequency of various types of system calls and constructing frequency vectors based on these statistics. Subsequently, an autoencoder is used to train and learn these frequency patterns. In the anomaly detection phase, by reconstructing these frequency vectors, if the reconstruction error exceeds a predetermined threshold, the system call sequence is determined to be an anomaly.

ReplicaWatcher [23]: This method first processes a continuous stream of events into a short monitoring interval (called a"Snapshot"). Next, feature vectors are generated from each snapshot to capture the differences between that snapshot and other snapshots. In the anomaly detection phase, the deviation of each snapshot from the ideal state (where all snapshots are identical) is evaluated. If any snapshot is found to have a significant deviation, it is judged to be abnormal.

Table VII summarizes key implementation differences. Our system (MADGuard) uniquely incorporates multi-service monitoring with Tetragon while requiring extended pre-training.

B. Detection Performance (RQ1)

Table VIII presents comprehensive detection metrics across three attack scenarios. Our analysis reveals three key patterns:

WebShell Attack Analysis: MADGuard achieves superior F1-score (0.8666) through optimal precision-recall balance, while CDL's exceptional accuracy (99.92%) stems from its frequency-based reconstruction mechanism that effectively filters common system call patterns. However, CDL's 41.26% missed detection rate (1 - 0.5874 Recall) exposes limitations in recognizing novel attack signatures. STIDE-BoSC's poor performance (F1=0.044) confirms the inadequacy of sliding window approaches for stealthy WebShell activities that exhibit temporal dispersion.

Code Injection Dynamics: While CHIDS demonstrates competitive F1-score (0.8881), its 19.64% false positive rate (1 - 0.8036 Precision) suggests over-sensitivity to parameter variations in PHP execution contexts. MADGuard's perfect precision demonstrates exceptional discriminative power in containerized PHP environments, accurately distinguishing legitimate Composer operations from malicious payload injections.

Information Leakage Characteristics: The proposed system's complete recall (1.0000) in Node.js environments validates our graph neural network's capability to detect low-and-slow data exfiltration patterns. CHIDS' precision-recall imbalance (1.0000 vs 0.4428) reveals inherent challenges in modeling asynchronous I/O operations characteristic of Node.js microservices.

Overall, the proposed method in this paper performs well in all scenarios, especially on F1-score and Accuracy, showing its high efficiency and reliability in anomaly detection, and the effectiveness of the proposed method in anomaly detection, compared with the existing anomaly detection system, the accuracy is improved by 15.07%. CDL does very well on Accuracy, but lacks Recall, which can lead to some under-reporting. CHIDS perform well in code injection scenarios, but Recall is low in information leak scenarios, affecting overall performance. ReplicaWatcher has high Precision in some scenarios but low Recall, which means it can generate false positives in different scenarios.

TABLE V
MICROSERVICE ATTACK ENVIRONMENT

Microservice Platform	Microservice	Application	CVE	Threat-Type	Attack Scenarios
TeaStore	webui	tomcat(8.5.19)	CVE-2017-12615	remote code execution	Webshell Attack
RobotShop	ratings	php(8.3.4-apache)	CVE-2024-2961	remote code execution	Code Injection
SockShop	frontend	node(8.5.0-alpine)	CVE-2017-14849	directory traversal	Information Leakage

TABLE VI
DISTRIBUTION OF EXPERIMENTAL DATA

Microservice Platform	Normal Node	Normal Edge	Abnormal Node	Abnormal Edge
TeaStore	4928	17791734	532	685002
RotbotShop	36709	5376687	1121	53572
SockShop	65146	10459454	60	211318

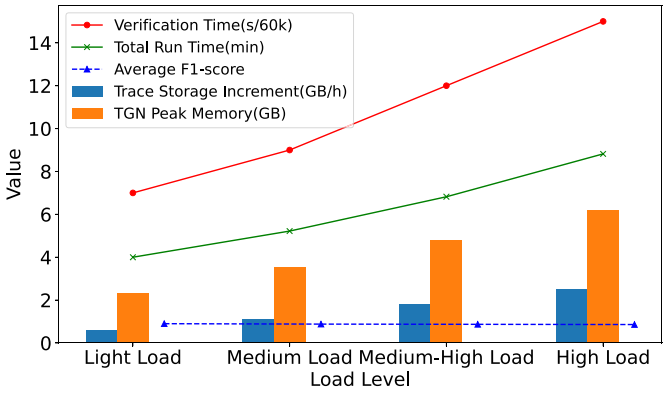

Fig. 5. MADGuard core component runtime overheads at different load levels.

C. Runtime Overhead & Scalability Analysis (RQ2)

The runtime comparison in Table IX reveals critical engineering trade-offs: *STIDE-BoSC's efficiency* (9s) comes at the cost of limited detection scope - its single-container focus ignores 30% of cross-service anomalies in our testbed; *CHIDS' prolonged runtime* (960s) stems from expensive graph isomorphism checks, increasing exponentially with sequence complexity; *MADGuard's moderate overhead* (240s) reflects the computational intensity of our hybrid architecture, but remains practical given its 1-minute detection interval. Combing Table VII, MADGuard's achieves 56% better F1-score than ReplicaWatcher with only 2.4× time increase.

To analyze the runtime overhead of MADGuard in the trace storage, TGN memory update, and forensic graph reconstruction phases, performance analysis was conducted under different load conditions. The experiment was based on a Kubernetes cluster environment verified in the paper, simulating varying degrees of load intensity by injecting real business traffic: CPU loads of 20% (light load), 40% (medium load), 60% (medium-high load), and 80% (high load), corresponding to peak business scenarios of Sockshop, Robotshop, and Teastore benchmark microservices. The collected log volume increased with the load and covered a mixed load pattern of complex attacks such as Web shell attacks and code injection, to meet the stress testing needs of real-world business scenarios.

The experimental results (Figure 5) show that the trace storage increment of the trace graph grows linearly with increasing load: although the storage increment per hour in the high load scenario is slightly higher than in the light load scenario, the overall growth is stable, with no nonlinear expansion observed. This performance is attributed to the optimized storage strategy of the trace graph, including hash coding techniques for compressing high-dimensional features and pod isolation mechanisms to avoid redundant cross-service data, proving its controllable storage overhead during long-term high load operation. At the same time, the peak memory of TGN remains stable across all load levels, without significant fluctuations with increasing load, due to the dynamic memory management strategy adopted by TGN, which focuses on recent key data and reduces the storage of the full historical log to efficiently control memory usage. Additionally, with increasing load, the graph reconstruction time per 60k data shows a slow growth trend: the lowest in the light load scenario, slightly higher in the high load scenario, but overall maintained at a low level within 14 seconds. This indicates that even in high load scenarios, the anomaly forensic process of MADGuard remains efficient, without a significant increase in time due to the increase in data volume, verifying the practicality of the forensic module under complex load conditions.

D. Forensic Analysis (RQ3)

To validate the effectiveness of the forensic module, performance analysis was conducted in three typical attack scenarios: Code Injection, Information Leakage, and WebShell Attack, with comparisons made to the Kairos [27] anomaly reconstruction method.

In the Code Injection attack scenario, MADGuard demonstrated high precision: apart from benign events frequently accessed within the service (such as '/etc/hosts') being misjudged as anomalies and occasional omissions of '/proc' file events associated with anomalies, core anomalous events (such as the creation of the malicious file '/shell.jsp') were all accurately identified. With the help of unique event identification and cross-service event reconstruction methods, the entire attack chain could be clearly and completely traced. In contrast, although Kairos' anomaly reconstruction method could detect some anomalies, it missed key events such as 'shell.jsp', and its community partitioning strategy fragmented the attack graph into multiple subgraphs, obscuring the relationships between anomalous events.

In the Information Leakage attack scenario, MADGuard effectively filtered out most false-positive events, with only a few '/proc' files being misjudged; similar to the Code Injection

TABLE VII
COMPARISON OF EXPERIMENTAL DETAILS OF ANOMALY DETECTION

Existing Anomaly Detection Methods	Detection Environment	Dataset Capture Tool/Format	Pre-training Required	Detection Interval
STIDE-BoSC	Single container	sysdig/json	Yes,875000 system call sequences	Ten system call windows
CHIDS	Single container	sysdig/scap	Yes,4-fold cross-validation 25% of the dataset for training	1 second
CDL	Single container	sysdig/json	Yes,three minutes of data	0.1 second
ReplicaWatcher	Microservices (multiple containers)	Sysdig and custom Sysdig eBPF filters/json	No	30 seconds
MADGuard	Microservices (multiple containers)	tetragon/json	Yes(four days of data)	1 minute

TABLE VIII
PERFORMANCE COMPARISON OF ANOMALY DETECTION SYSTEM

Scenarios	Existing Anomaly Detection Methods	Precision	Recall	F1-score	Accuracy
Webshell Attack (TeaStore)	MADGuard	1.0000	0.7647	0.8666	0.9298
	ReplicaWatcher	1.0000	0.3750	0.5454	0.8250
	STIDE-BoSC	0.9504	0.0225	0.0440	0.6811
	CDL	0.9987	0.5874	0.7397	0.9992
	CHIDS	0.9894	0.3916	0.5545	0.8474
Code Injection (RobotShop)	MADGuard	1.0000	0.8000	0.8888	0.9166
	ReplicaWatcher	0.6666	1.0000	0.8000	0.8333
	STIDE-BoSC	0.7486	0.8532	0.7975	0.6750
	CDL	0.7786	0.6628	0.7161	0.9993
	CHIDS	0.8036	0.9925	0.8881	0.9275
Information Leakage (SockShop)	MADGuard	0.9677	1.0000	0.9836	0.9761
	ReplicaWatcher	0.8068	0.8605	0.8328	0.7195
	STIDE-BoSC	0.8643	0.1822	0.3009	0.3628
	CDL	0.8366	0.7014	0.7631	0.9972
	CHIDS	1.0000	0.4428	0.8321	0.6138

scenario, the frequent access to '/etc/hosts' within the service was misjudged as an anomaly, but core anomalous events (such as sensitive accesses to '/etc/passwd' and '/etc/fstab') were all successfully detected, and the attack path could be completely reconstructed. Kairos performed poorly in this scenario, not only misjudging a large number of normal events as anomalies but also disrupting the coherence of the attack chain due to its graph partitioning strategy.

In the WebShell Attack scenario, MADGuard significantly filtered out false-positive events through top-k event filtering, achieving a 33% increase in precision compared to Kairos, with only a 3% decrease in recall, thus balancing precision and completeness.

In terms of throughput performance, MADGuard demonstrated efficient processing capabilities: it processed 26,672 events in the Code Injection scenario in just 4 seconds, handled 118,784 events in the Information Leakage scenario in 7 seconds, and processed 452,608 events in the WebShell Attack scenario in 10 seconds. On average, it could handle over 60,000 events every 7 seconds. Compared to Kairos' real-time attack reconstruction capability, MADGuard's event processing rate per second increased by more than 7,000. The specific performance metrics are shown in Table X.

To further illustrate the core value of the security forensics module, this section starts from the actual operational needs of security analysts, taking the WebShell attack scenario in the TeaStore environment as an example, and compares the differences in the forensic results of "MADGuard without a forensic module," "existing Kairos forensic method,"

TABLE IX
RUNTIME OVERHEAD COMPARISON

Methods	MADGuard	ReplicaWatcher	STIDE-BoSC	CDL	CHIDS
Runtime(s)	240s	100s	9s	70s	960s

and "MADGuard with a forensic module" (the relevant results are shown in Figure 6), with a detailed analysis as follows:

When MADGuard is not integrated with a forensic analysis module (Figure 6a)), the system can only present the detected anomalous nodes in isolation, without the ability to correlate the attack propagation paths between nodes, nor to label the microservices to which the anomalous nodes belong (for example, it cannot distinguish whether the anomalous nodes are from the "Order Service" or the "Payment Service"), causing security analysts to manually investigate the relationships and affiliations of nodes, greatly increasing the complexity and time consumption of attack tracing.

Regarding the existing Kairos forensic method (Figure 6b)), although it can connect some anomalous nodes to form paths, there are two key shortcomings: First, Kairos uses a community division algorithm for path aggregation, which is prone to splitting cross-service attack propagation chains into several isolated subgraphs (for example, the complete path of a WebShell attack from the "Frontend Service" to the "Database Service" is split), failing to reconstruct the global propagation process of the attack; Second, due to the possibility of similar names or paths of nodes across different services

TABLE X
PERFORMANCE COMPARISON OF MADGUARD AND KAIROS IN DIFFERENT ATTACK SCENARIOS

Scenario	Method	Precision	Recall	F1-Score	Throughput (Events/s)	Completeness of Attack Event Reconstruction	Clarity of Attack Path Reconstruction
Code Injection	MADGuard	0.9780	0.7647	0.8666	7200	✓	✓
	Kairos	0.8289	0.5294	0.6461	5700	×	×
Information Leakage	MADGuard	0.8181	0.9310	0.8709	17000	✓	✓
	Kairos	0.2903	0.9310	0.4426	14800	×	×
WebShell Attack	MADGuard	0.9961	0.9086	0.9503	30100	✓	✓
	Kairos	0.6666	0.8681	0.7542	45200	×	×

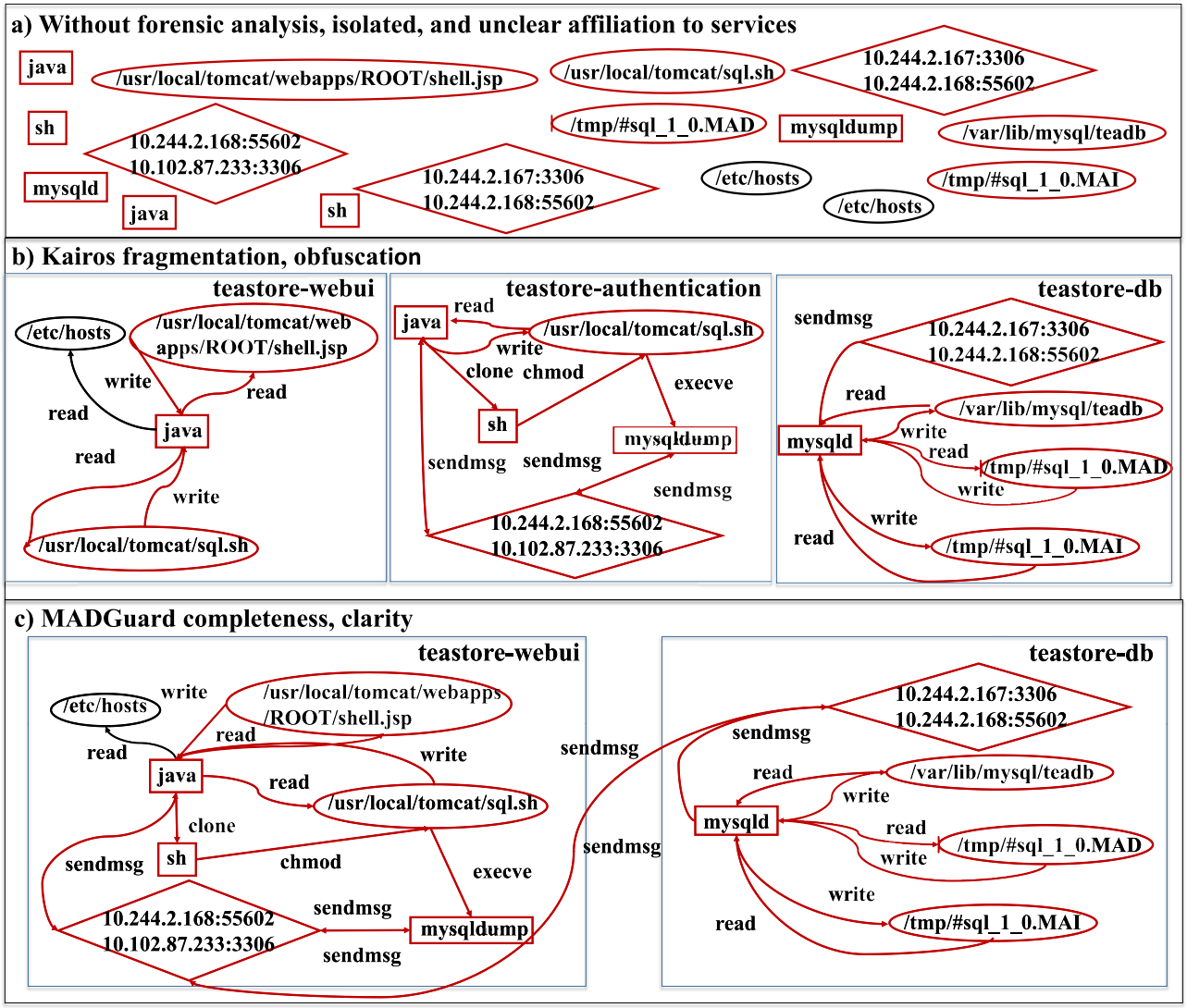

Fig. 6. Comparison of forensic analysis methods in WebShell attack: impact on attack path reconstruction and node affiliation clarity.

in a microservices environment (for example, each service contains a "log.txt" file node), Kairos lacks a mechanism for clearly identifying node affiliations, which can lead to analysts mistakenly identifying nodes under different services as belonging to the same service, resulting in misjudging the scope of the attack.

In contrast, MADGuard's forensic analysis module addresses the aforementioned issues through two core designs: On one hand, by introducing a unique service identifier (mid), it labels each node with explicit service affiliation information (for example, "order-service:node-123"), helping analysts quickly locate the service to which the anomalous node belongs; On the other hand, relying on a cross-service attack chain identification mechanism, it can associate isolated anomalous nodes beyond service boundaries, completely reconstructing the global propagation path from the attack entry point (such as the WebShell implantation point) to the target service (such as the database data theft point)

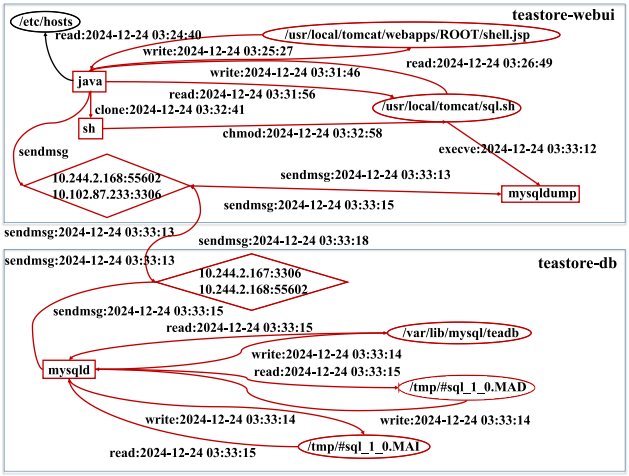

Fig. 7. Forensic analysis of Webshell attack in TeaStore.

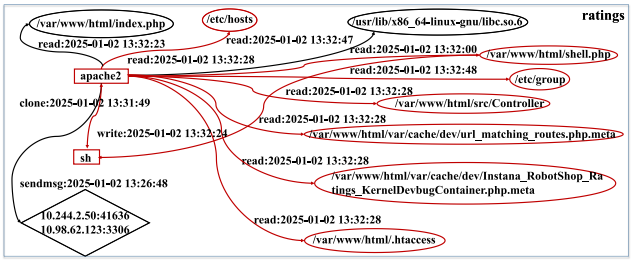

Fig. 8. Forensic analysis of code injection in RobotShop.

(Figure 6c)). Ultimately, it provides security analysts with forensic results that are "clear in node affiliation and complete in path continuity," significantly reducing the difficulty and time cost of attack tracing.

MADGuard Forensic Analysis Results:

WebShell Attack (TeaStore): MADGuard detected edges related to the *shell.jsp, sql.sh*, and mysqldump attack nodes in the teastore-webui service environment. These edges exhibited significant reconstruction errors compared to benign edges. Similarly, in the teastore-db service environment, MADGuard identified unusual teadb files and IP information communicating with the webui service.

Code Injection (RobotShop): During the attack, the attacker generated access to service metadata in the RobotShop ratings service, including *KernelDevbugContainer.php.meta, urlmatchingroutes.php.meta*, and *.htaccess*. By identifying malicious edges associated with these nodes, MADGuard traced the malicious nodes and their related chain information to reconstruct the entire attack path.

Information Leakage (SockShop): The attacker left traces of accessing sensitive files in the front-end service. MADGuard identified abnormal edges related to sensitive files through a reconstruction error and used IDF to assist in identifying abnormal nodes. By tracing the one-hop events related to these abnormal edges and nodes, MADGuard reconstructed the attack path.

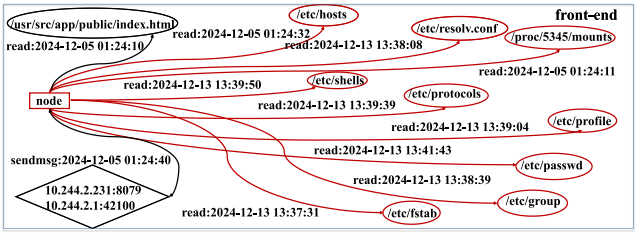

Fig. 9. Forensic analysis of Information Leakage in SockShop.

E. Comparison of MADGuard-TGN With Existing Temporal Models

To verify the superiority of the TGN (Temporal Graph Network) adopted by MADGuard in microservices anomaly detection scenarios, this paper designs comparative experiments in three typical microservices benchmark environments: TeaStore, SockShop, and RobotShop. The data preprocessing procedures (hash encoding, positional encoding), feature dimensions (each 16-dimensional), and evaluation protocols (with Precision, Recall, and F1-score as core metrics) are kept completely consistent, with only the core temporal modeling module TGN being replaced by existing temporal graph models TGAT (Temporal Graph Attention Network) and EvolveGCN (Evolving Graph Convolutional Networks) for a fair comparison of the models' adaptability.

The experimental results (Figure 10) show that TGN exhibits better detection performance in all three types of microservices attack scenarios, especially in cross-service and highly dynamic scenarios: in the RobotShop code injection scenario, TGN's F1-score reaches 0.8888 and Precision maintains 1.0, accurately distinguishing normal dependency calls in the PHP environment from malicious code injection, while TGAT and EvolveGCN's F1-scores are only 0.8177 and 0.7615, respectively; in the TeaStore WebShell attack scenario, TGN's F1-score is 0.8666, although TGAT's F1-score (0.8809) is slightly higher, TGAT's Precision (0.8916) is significantly lower than TGN's (1.0), with more normal events being misjudged; in the SockShop information leakage scenario, TGN's Recall reaches 1.0000, fully capturing the low-frequency sensitive file access behavior in Node.js asynchronous I/O, while TGAT and EvolveGCN's Recall are 0.8881 and 0.7926, respectively, easily omitting local temporal anomalies. Overall, TGN's average F1-score in the three scenarios reaches 0.9130, improving by 3.87% over TGAT (0.8743) and 13.51% over EvolveGCN (0.7779), with better balance between Precision and Recall, better meeting the actual needs of microservices anomaly detection for "low false positives, low false negatives."

Further analysis of the core reasons for the performance differences between models: Although TGAT strengthens the temporal correlation weights between nodes through temporal attention mechanisms, in microservices scenarios, its attention allocation is easily disturbed by high-frequency normal events (such as inter-service heartbeat calls, routine business requests)—for example, in the TeaStore WebShell attack, the normal communication frequency between the webui service

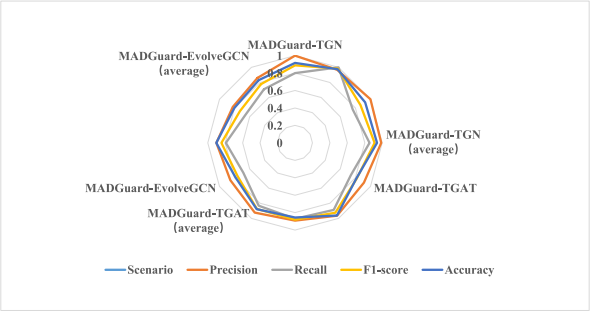

Fig. 10. Comparison of model performance across scenarios.

TABLE XI
COMPARISON OF ABLATION COMPONENTS PERFORMANCE

Ablation Component	Scenario	Precision	Recall	F1-Score	Accuracy	AUC
Without Memory	Robotshop	0.625	1.0	0.7692	0.9189	0.9531
	Sockshop	0.9615	0.8620	0.9090	0.8809	0.8925
	Teastore	0.8181	0.6923	0.7500	0.9473	0.8362
Without Embedding	Robotshop	0.5555	1.0	0.7142	0.8918	0.9375
	Sockshop	0.9523	0.6896	0.7999	0.7619	0.8063
	Teastore	1.0	0.6923	0.8181	0.9649	0.8461
Without Encoder	Robotshop	0.7142	1.0	0.8333	0.9459	0.9687
	Sockshop	0.9062	1.0	0.9508	0.9285	0.8846
	Teastore	1.0	0.6428	0.7826	0.9561	0.8214
Without Decoder	Robotshop	0.7899	1.0	0.8826	0.9090	0.9146
	Sockshop	0.9354	1.0	0.9666	0.9523	0.9230
	Teastore	1.0	0.6923	0.8181	0.9649	0.8461
Without Multi-dimensional Message Fusion	Robotshop	0.4166	1.0	0.5882	0.4166	0.5
	Teastore	0.7142	1.0	0.8333	0.7142	0.5
	Sockshop	0.2982	1.0	0.4594	0.2982	0.5
Without Temporal Fusion	Robotshop	0.4166	1.0	0.5882	0.4166	0.5000
	Sockshop	0.9666	0.9666	0.9666	0.9523	0.9416
	Teastore	0.2982	1.0	0.4594	0.2982	0.5000

and the db service (50 times per second) is much higher than the malicious data theft frequency (once per second), causing TGAT's attention weights to tilt towards normal events, resulting in decreased sensitivity to low-frequency anomalies; EvolveGCN relies on fixed-frequency updates (default every 30 seconds) of graph convolutional kernels to adapt to temporal changes, unable to cope with the rapid topology adjustments caused by microservices dynamic scaling—when SockShop's frontend service scales from 2 to 5 Pods due to traffic fluctuations, EvolveGCN's kernel updates lag behind the topology changes, leading to broken cross-service event associations and significant performance degradation (F1-score drops by 8.2%).

In contrast, TGN's adapted design for microservices characteristics (distributed tracing graph's mid/podname identifiers, memory-enhanced node state updates, dynamic time window temporal aggregation) effectively avoids the above issues: its memory-enhanced mechanism maintains the causal dependencies of long-term cross-service attacks (such as the 10-minute attack chain temporal modeling in WebShell attacks), and the dynamic time window (60 seconds) balances real-time performance with event completeness, both working together to keep TGN's stable detection performance in highly dynamic and cross-service scenarios of microservices, verifying the rationality of TGN as the core temporal modeling module.

F. Ablation Study (RQ4)

To verify the specific roles of each component in MADGuard, we designed an ablation study using the controlled variable method, with the results shown in Table XI. By systematically removing model components and observing changes in performance, the core functions of each component and their impact on the model's effectiveness can be clearly demonstrated: after removing the Memory module, the model loses its ability to dynamically capture and update node and edge information, leading to confusion between benign and anomalous event features, and a significant increase in the false positive rate; without the Embedding module, edge embeddings rely on static historical node information and fail to capture dynamic changes in nodes, making it difficult to distinguish subtle differences in events, leading to a large number of false positives; the Encoder module, as the core integrator of memory and embeddings, when removed, causes the model to lose its ability to dynamically learn the evolution of graph structures, resulting in decreased precision in anomaly detection and an increased false positive rate; and after removing the Decoder module, the model is unable to explore long-term dependencies between nodes, and since anomalous events often manifest through the accumulation of long-term features, this ultimately leads to a significant increase in the false negative rate.

In addition to the synergistic effect of the core components, the model's anomaly detection capability also relies on the support of a multidimensional information fusion mechanism. Among them, Multidimensional message fusion has been proven to be a core mechanism for effective anomaly detection—by integrating multi-dimensional message features, the model's AUC value is significantly improved from a baseline level of 0.5, providing a more comprehensive feature basis for anomaly identification. The importance of Temporal fusion, however, is scenario-dependent: in Webshell detection, temporal fusion is crucial, supporting the model to achieve a 100% recall rate (although the precision is 29.8%); it also plays a key role in code injection identification; while the information leakage detection scenario demonstrates the independence of temporal patterns, indicating that the necessity of temporal fusion is relatively lower in such scenarios.

In summary, MADGuard ensures the capture and integration of dynamic graph features through its core components, combined with a multidimensional message fusion and scenario-adaptive temporal fusion mechanism, together achieving a comprehensive improvement in anomaly detection performance: the core components address the issues of dynamic feature learning and long-term dependency capture, while the fusion mechanism enhances feature discrimination capability through multi-dimensional and temporal information, both synergistically supporting the model's effectiveness.

To further verify the model's long-term adaptability in dynamic scenarios, we designed comparative experiments for the Online Update mechanism. One group was the MADGuard without OU, which did not incorporate the OU mechanism, and we observed the evolution of its performance over time steps; the other group was the MADGuard with OU,

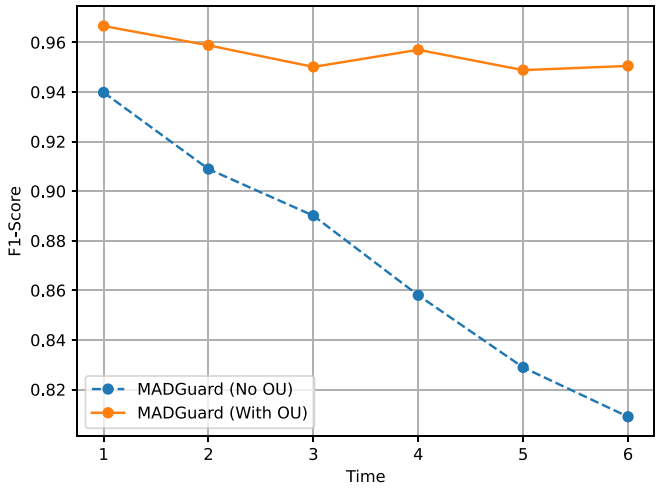

Fig. 11. F1-Score Comparison with and without OU.

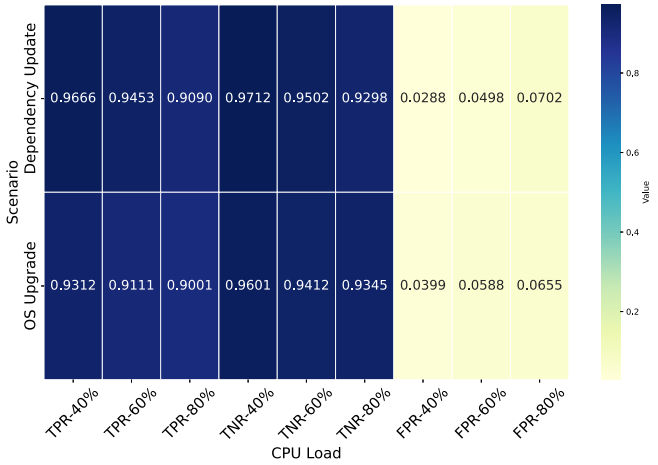

Fig. 12. TPR, TNR, and FPR for different scenarios and CPU loads.

which integrated the OU mechanism, and we analyzed the trend of its performance over time. The experimental results (Figure 11) showed that without the OU mechanism, the model's performance degraded significantly over time, with the F1-score dropping sharply from 96% to 80% within just six time steps; after introducing the OU mechanism, the model could continuously learn new event patterns, and the F1-score was essentially stable, maintaining around 96%. This indicates that the OU mechanism can effectively overcome the performance degradation caused by temporal evolution, allowing the model to maintain stable anomaly detection capabilities over the long term, and working in conjunction with core components and fusion mechanisms to jointly enhance the lifecycle performance of MADGuard.

G. Robustness Experiments

In a microservices environment, the dynamic evolution of business loads (such as resource consumption changes) and updates to benign behavior (such as component upgrades) are common occurrences. These changes can easily be misjudged as anomalies by anomaly detection systems, leading to increased false positive rates or decreased detection accuracy. To verify the robustness of MADGuard under such scenarios—namely, its "resistance to interference" against benign behavior changes and its stable detection capability for genuine anomalies—we designed synthetic stress environment experiments that closely mimic real-world operational scenarios and evaluated its performance using key metrics such as TPR (True Positive Rate), TNR (True Negative Rate), and FPR (False Positive Rate).

Experiment Design focuses on two typical benign dynamic changes in a microservices environment: pod configuration updates and load fluctuations. Regarding pod configuration updates, we selected common base component upgrade scenarios encountered in microservices operations: one is the base OS image update, upgrading the shipping service of the robotshop application from Debian 10-buster to Debian 11-bullseye, simulating version iteration of the underlying operating environment; the other is dependency package updates, upgrading the libthrift library dependency of the recommender service of the teastore application from version 0.16.0 to 0.20.0, simulating business component dependency upgrades. Both types of operations are benign changes without malicious intent but may cause short-term fluctuations in service communication patterns and resource consumption characteristics, potentially interfering with the detection system.

To further simulate the combined effects of load and configuration changes in a real environment, the experiment repeated the above pod update operations under different CPU load pressures: controlling the system CPU load occupancy rate to 40% (light load), 60% (medium load), and 80% (high load), covering typical load ranges in daily microservices operations. Through this design, a comprehensive evaluation of MADGuard's stability under "configuration change + load fluctuation" compound scenarios is possible.

Experiment Results (as shown in Figure 12) indicate that MADGuard maintained excellent robustness in various synthetic stress environments: for benign behavior changes (such as OS image upgrades, dependency package updates), its true positive rate (TPR) remained consistently above 90%, ensuring effective detection of genuine anomalies; at the same time, the false positive rate (FPR) was controlled below 8%, without generating a large number of false alarms due to configuration updates or load fluctuations. This performance is attributed to MADGuard's core technical mechanisms: a sliding window dynamically capturing the latest baseline of benign behavior, combined with an online update mechanism to adjust the feature distribution in real-time, enabling the model to quickly adapt to normal evolution of business, rather than misjudging benign changes as anomalies.

To verify the robustness of MADGuard in cross-microservices scenarios, we designed a cross-environment training-testing experiment: we selected different microservices environments and trained the model in one environment, then migrated it to another for testing. Specifically, we set up two groups of experiments: one trained in the robotshop environment and tested in the teastore environment; the other trained in the sockshop environment and tested in the robotshop environment. The experimental results (as shown

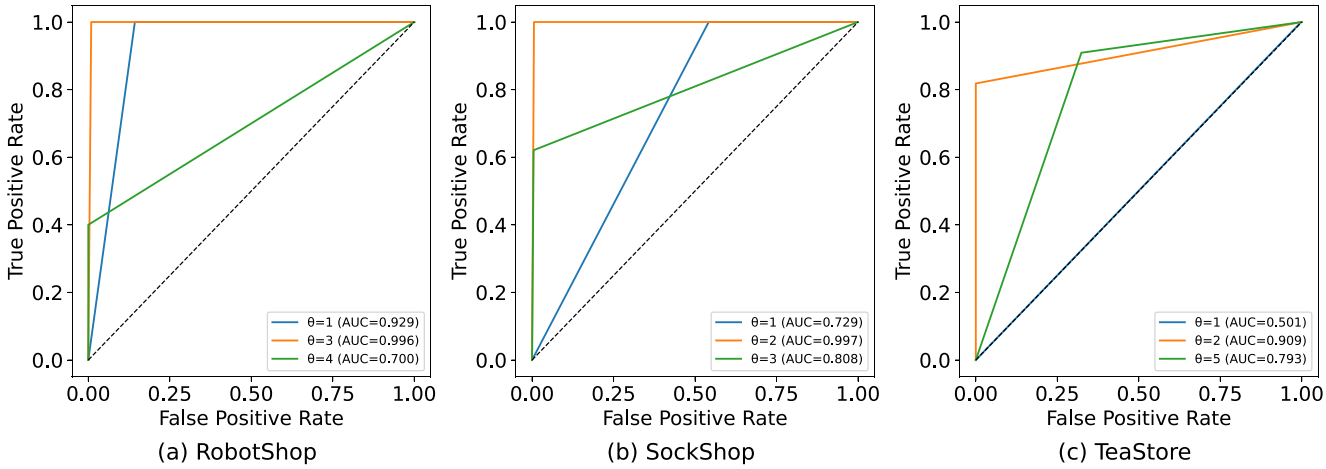

Fig. 13. $|\theta|$.

TABLE XII
CROSS-ENVIRONMENT TRAINING AND TESTING RESULTS

Train	Test	Precision	Recall	F1-score	Accuracy
robotshop	teastore	1.0	0.7923	0.8841	0.9649
sockshop	robotshop	0.8333	1.0	0.9090	0.9677

in Table XII) indicate that although there was a performance fluctuation of about 1% during cross-environment testing, the F1-score metric remained consistent with the performance of training and testing within the same environment. For instance, when trained in robotshop and tested in teastore, the precision reached 1.0, recall was 0.7923, and the F1-score was 0.8841; when trained in sockshop and tested in robotshop, the precision was 0.8333, recall was 1.0, and the F1-score was 0.9090. This demonstrates that MADGuard can stably maintain detection performance in cross-microservices system scenarios, fully verifying its cross-system robustness and its ability to effectively adapt to microservices environments with different loads and configurations.

H. Alert Threshold and Sliding Window Optimization

In this section, we provide a detailed description of the optimization process for two key parameters in MADGuard: the alert threshold and the sliding window.

Alert Threshold Optimization. During the optimization of the alert threshold, we first performed a preliminary evaluation using benign data that had not been trained on. The distribution of anomaly scores of benign data was observed to have scenario-specific characteristics across the three microservice scenarios. In the RobotShop environment, the anomaly scores of benign data were concentrated within 3, while in the SockShop and TeaStore environments, these scores were stable around 2. Based on these observations, we set different alert thresholds (θ) within the range of 1 to 5 and evaluated their rationality using the ROC curve and AUC value (a core indicator of the model's ability to distinguish between benign and abnormal events). The results are shown in Figure 13.

In the RobotShop environment, when the threshold θ was set to 1, it was below the general anomaly score range of benign data ($\leq$3), which led to a large number of benign data being misclassified as anomalies. Although the AUC was 0.929, the false alarm rate was excessively high. When θ was set to 4, the threshold exceeded the upper limit of benign data, causing some abnormal data to be missed due to insufficient scores, resulting in a significant increase in the miss rate and a sharp drop in AUC to 0.700. However, when θ was set to 3, the AUC reached the highest value of 0.996, indicating that the model could most accurately distinguish between benign and abnormal events at this point, effectively controlling false alarms while avoiding misses. Therefore, θ =3 was determined to be the optimal threshold for this environment.

In the SockShop microservice environment, the anomaly scores of benign data were concentrated around 2. When the threshold θ was set to 1 (below 2), a large number of benign data was misclassified as anomalies, leading to a surge in false alarms and an AUC of only 0.729. When θ was set to 3 (above 2), some abnormal data were missed due to insufficient scores, resulting in a significant increase in misses and an AUC drop to 0.808. However, when θ was set to 2, the AUC reached the highest value of 0.997, indicating the optimal distinguishing ability of the model. Thus, $\theta = 2$ was established as the optimal threshold for this environment.

For the TeaStore microservice environment, the anomaly scores of benign data were also concentrated around 2. When θ was set to 1, the model's distinguishing ability was close to random (AUC = 0.501), resulting in a large number of misses. When the threshold was set too high (e.g., θ = 5), the AUC dropped to 0.793, with an increased risk of false alarms. However, when θ was set to 2, the AUC reached the highest value of 0.909, indicating that the model could effectively balance false alarms and misses. Therefore, $\theta = 2$ was determined to be the optimal threshold for this environment.

Sliding Window Optimization. To verify the effectiveness of the model's default τ = 1-minute (60s) time window setting, we compared the performance of 30s (a shorter window) and 90s (a longer window) in the RobotShop, SockShop, and TeaStore microservice environments. We evaluated the impact of window length on detection performance using four

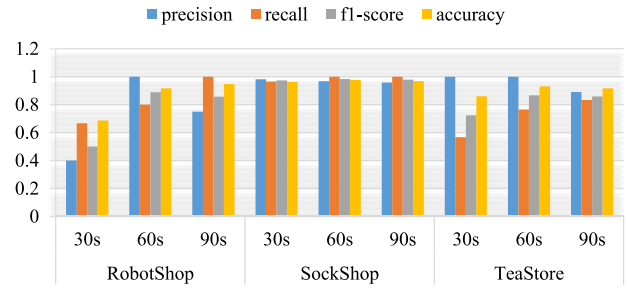

Fig. 14. $|\tau|$.

core metrics: precision, recall, f1-score, and accuracy. The experimental results are shown in Figure 14.

From the trends of the metrics, it was evident that the shorter 30s window exhibited significant drawbacks in all three environments. Since attackers often exhibit latent behavior in microservice environments, the features of attack events are dispersed across multiple short windows, resulting in a lower proportion of abnormal features within a single window. The data showed that in the RobotShop environment, the recall dropped to around 0.6 with the 30s window, while in the SockShop environment, the recall was less than 0.7. The composite f1-score was approximately 0.65 in the RobotShop environment and around 0.7 in the SockShop environment, both significantly lower than the 60s window level. This indicates that the 30s window is prone to missing key abnormal events due to feature fragmentation.

However, the longer 90s window, although capable of covering more potential attack event sequences, also included too many normal events, resulting in a lower feature purity within the window. The experimental data showed that in the 90s window, the precision in all three environments declined. In the RobotShop environment, the precision dropped to around 0.8, and in the TeaStore environment, it fell below 0.75. The increased misclassification of normal events as anomalies directly led to a lower f1-score in the 90s window compared to the 60s window in all three environments.

In contrast, the 60s window demonstrated optimal performance across all metrics. In the RobotShop environment, the f1-score reached 0.95, with precision and recall both stable above 0.9. In the SockShop environment, the f1-score was approximately 0.9, with precision and recall maintained above 0.85. In the TeaStore environment, the f1-score was the highest, reaching 0.92. This indicates that a 1-minute window can effectively aggregate the continuous features of attack events while controlling the excessive mixing of normal events, fully adapting to the latent nature and feature continuity requirements of attack behavior in microservice environments.

I. Impact of Hash Encoding and Positional Encoding on Model Performance

To clarify the dimensions of hash and positional encodings, the risk of hash collisions, and the stability of the encoding scheme when new data types appear, we conduct three targeted experiments to systematically validate the rationality and robustness of our encoding mechanism. All experiments are carried out on the TeaStore, SockShop, and RobotShop microservice benchmarks, while the core model parameters—TGN structure, sliding-window τ, and anomaly threshold θ remain unchanged; only encoding-related variables are manipulated.

TABLE XIII
EXPERIMENTAL RESULTS OF HASH CODING DIMENSIONS

Hash Dimension (H)	Scenario	Precision	Recall	F1-score	Hash Collision Rate
8	TeaStore	0.9211	0.9423	0.9316	21.89%
	SockShop	0.9714	0.7917	0.8730	27.58%
	RobotShop	0.9375	0.7308	0.8211	24.97%
16	TeaStore	0.9677	1.0000	0.9836	11.75%
	SockShop	1.0000	0.8000	0.8888	15.21%
	RobotShop	1.0000	0.7647	0.8666	9.64%
32	TeaStore	0.9712	1.0000	0.9855	5.82%
	SockShop	1.0000	0.8125	0.8965	8.73%
	RobotShop	1.0000	0.7731	0.8723	4.21%

TABLE XIV
EXPERIMENTAL RESULTS OF POSITION CODING DIMENSIONS

Position Dimension (P)	Scenario	Precision	Recall	F1-score	Anomaly Path Clarity (1-5)
8	TeaStore	0.9356	0.9231	0.9293	3 (Partial node ambiguity)
	SockShop	0.9643	0.7857	0.8667	2 (Cross-service breakage)
	RobotShop	0.9524	0.7500	0.8372	2 (Local spatial loss)
16	TeaStore	0.9677	1.0000	0.9836	5 (Complete position info)
	SockShop	1.0000	0.8000	0.8888	5 (Traceable cross-service)
	RobotShop	1.0000	0.7647	0.8666	5 (Clear anomaly spread)
32	TeaStore	0.9689	1.0000	0.9844	5 (No difference from 16)
	SockShop	1.0000	0.8050	0.8902	5 (No difference from 16)
	RobotShop	1.0000	0.7680	0.8690	5 (No difference from 16)

1) Encoding-Dimension Hyper-Parameter Study: We determine the optimal dimensions for hash encoding (H) and positional encoding (P) through two independent-variable experiments.**Hash-dimension experiment:** Fix positional encoding at 16-D, vary hash dimensions in {8, 16, 32}, and evaluate detection metrics (Precision, Recall, F1-score) together with the hash-collision rate.**Positional-dimension experiment:** Fix hash encoding at 16-D, vary positional dimensions in {8, 16, 32}, and evaluate detection performance plus the completeness of positional information (measured by anomaly-path reconstruction clarity on a 1–5 scale, 5 being best). Tables XIII and XIV summarize the results. The optimal configuration is 16-D for both hash and positional encodings.

Hash-dimension results: 8-D hash encoding yields a collision rate above 20% due to the compact space, confusing normal and abnormal node features. 32-D hash encoding reduces collisions below 9%, but F1-score improves by only 0.5%–2% while doubling storage and computation cost, offering poor cost-effectiveness.**Positional-information results:** 16-D positional encoding fully captures node spatial relationships, achieving the maximum clarity score (5). 8-D encoding suffers from insufficient capacity, causing cross-service path discontinuities; 32-D yields no additional gain and only increases computational complexity.

2) Hash-Collision Risk Analysis: We localize collision sources and quantify their impact in two steps. (1) **Collision source localization:** After deduplication of logs from the three scenarios, we hash all nodes (process paths, file directories, network IPs, etc.) and identify colliding original log entries.

TABLE XV
STATISTICS OF HASH COLLISION SOURCES

Scenario	Total Collision Samples	Dominant Collision Source	Proportion of This Source	Collision Data Feature
TeaStore	1286	/proc/pid directory logs	78.3%	Similar structure (only PID differs)
SockShop	1562	/proc/pid directory logs	82.1%	Similar structure (only PID differs)
RobotShop	1143	/proc/pid directory logs	75.6%	Similar structure (only PID differs)

TABLE XVI
IMPACT OF HASH COLLISIONS ON MODEL RECOGNITION ABILITY

Scenario	Anomaly Proportion in Collision Samples	Recognition Accuracy (Non-collision)	Recognition Accuracy (Collision)
TeaStore	0.3%	98.7%	98.5%
SockShop	0.5%	88.9%	88.6%
RobotShop	0.4%	86.7%	86.5%

Common patterns are analyzed. (2) **Collision impact verification:** Log entries responsible for collisions are labeled 'collision samples.' We compute their proportion among normal and abnormal events, and compare the anomaly-detection accuracy of collision vs. non-collision samples.

Tables XV and XVI present the results. Roughly 80% of collisions stem from Linux `/proc/pid` directory accesses: the kernel creates a unique `/proc/pid` for every process; logs differ only in PID, causing high structural similarity and frequent hash collisions. Collision samples account for less than 0.5% of abnormal events, and the anomaly-detection accuracy difference between collision and non-collision samples is below 0.3%. Thus, `/proc/pid` logs are redundant for anomaly discrimination, and hash collisions do not impair the model's core capability.

3) Encoding Stability With New Data Types: To simulate production scenarios in which new data types emerge, we design the following protocol: (1) **Add new data types:** We augment the original logs with two common production sources—container-resource monitoring logs (memory usage, container start/stop events) and microservice API-call logs (response time, success rate). (2) **Stability metrics:** Using the existing 16-D hash + positional encodings without any parameter tuning, we compare model performance (Precision, Recall, F1-score) between 'original only' and 'original + new data' inputs. Performance fluctuation within 2% is deemed stable.

Table XVII shows results. After adding both new data types, F1-score varies by less than 0.7%, with no significant drop in Precision or Recall. This indicates that the 16-D encoding scheme does not rely on prior knowledge of specific data types. The hash function compresses high-dimensional features of new data into a 16-D vector via uniform mapping, while positional encoding supplements structural information through spatial relationships, requiring no re-tuning to accommodate new data types.

J. Deployment

MADGuard is deployed in a continuous delivery environment using containerization, with Tertagon collecting event stream information in real-time. When the real-time event stream detection results deviate from the historical baseline, the system automatically triggers continuous integration tools (such as CircleCI) to initiate the Online Update (OU Section IV-D) mechanism. To avoid affecting other services, the OU training process is encapsulated and executed within a dedicated Docker container (MADGuard-retrain). Inside MADGuard-retrain, modules for data preprocessing, model training, and weight updates are pre-installed, completing the model weight update by inputting historical window data and new window data. Once the new model is generated, it immediately triggers a containerized deployment at the service layer. The model service container (MADGuard-service) dynamically loads the model through a shared storage volume and initiates an automated verification process. First, A/B testing is conducted in a shadow environment, comparing the new model's anomaly detection accuracy and false positive rate with the old model under real-time logs. After verification, a canary deployment is performed, using Tertagon to route 5% of production logs to the new version container, while the Prometheus monitoring module collects performance metrics in real-time. If key business metrics (false positive rate threshold) do not exhibit anomalies, the Kubernetes rolling update strategy is initiated, gradually replacing old version containers until full coverage is achieved.

TABLE XVII
CODING STABILITY COMPARISON UNDER NEW DATA TYPES

Scenario	Data Input Type	Precision	Recall	F1-score	Performance Fluctuation (vs. Original)
TeaStore	Original Data Only	0.9677	1.0000	0.9836	-
	Original + New Data	0.9652	0.9920	0.9785	-0.52%
SockShop	Original Data Only	1.0000	0.8000	0.8888	-
	Original + New Data	0.9960	0.7950	0.8832	-0.63%
RobotShop	Original Data Only	1.0000	0.7647	0.8666	-
	Original + New Data	0.9980	0.7600	0.8618	-0.55%

VI. DISCUSSION

Generality Challenges of Microservice Anomaly Detection Systems. Existing microservice anomaly detection methods typically require model retraining for different microservice environments to ensure accurate identification of abnormal events. With the widespread adoption of microservices, diverse operation modes are employed across various microservice systems, and system log information also varies. This necessitates training models from scratch for different microservice systems, making it difficult for existing anomaly detection models to achieve the "compile once, run everywhere" goal.

Public Dataset Availability. Currently, in the field of microservice anomaly detection, public datasets either do not exist or contain only benign data [40], which lack practical reference value. The lack of a unified public dataset makes it difficult to evaluate and compare the performance of different anomaly detection methods.

Concept Drift. Concept drift is a common problem in machine learning models [41], [42]. With the evolution of microservice systems, new components or services continue to appear, and anomaly detection models may misidentify new normal patterns as anomalies, resulting in false positives. Usually, the way to solve this problem is to update the training data set and retrain the anomaly detection model. Although

the anomaly detection system proposed in this paper can update the detection model efficiently, it becomes impractical to update the model manually over time. Therefore, how to realize the automatic update of anomaly detection model is an important challenge.

Ensuring the Integrity of Logs. The log sources of our system are based on Tetragon, a Kubernetes observation tool built on eBPF technology. The core advantage of Tetragon lies in its ability to dynamically insert and remove probes directly in the kernel space, which greatly reduces the latency and risk of log loss that may be introduced by traditional log collection methods. However, we fully understand the challenges that log integrity may face in real-world environments. In response to the issue of log loss, existing research primarily focuses on log compression techniques, including lossless and lossy compression [36], [43], [44], [45]. Lossless compression technology reduces the size of log data by identifying and eliminating redundant information within it, having minimal impact on the performance of subsequent anomaly detection models based on log sequences. Lossy compression techniques, in pursuit of higher compression ratios or processing efficiency, actively discard some log data that is considered to have lower value or higher redundancy. This method of compression can affect the performance of anomaly detection models to varying degrees, depending on the amount and strategy of the discarded log data. We plan to focus on and test the robustness of our anomaly detection model under different types and degrees of log loss in future research, and explore the integration of lossless compression technology to enhance detection efficiency.

Zero-day/Unknown Anomaly Detection. Zero-day and unknown anomalies represent emerging patterns of malicious behavior that do not yet have established signatures or labeled datasets. Traditional security systems, which rely on predefined rules and signature databases, are inherently limited in their ability to detect such novel threats [46], [47]. MADGuard, however, is designed to operate without the need for prior knowledge of attack patterns. It is trained on benign data and employs edge reconstruction errors to identify behaviors that deviate from the norm. This approach is particularly well-suited for the detection of unknown or zero-day anomalies, as it does not rely on pre-existing attack signatures.

Limitations of Existing Distributed Tracing Technologies. In response to the challenges of large-scale distributed systems, Google has developed large-scale tracing tools such as Dapper [48]. Widely adopted, Dapper has given rise to a range of derivative tools, including Zipkin, Jaeger, osprofiler, and opentracing. Building on these tracing tools, researchers have designed intelligent anomaly detection solutions for microservices in spatiotemporal and edge computing environments [49], [50], [51]. However, existing tracing tools (such as Dapper) primarily focus on data at the application layer of microservices (e.g., API call chains, latency information) and are unable to capture the characteristics of external attackers targeting microservices (for instance, the malicious file *sql.sh* in a WebShell attack). As a result, these tools are limited in their ability to detect external attacks and struggle to effectively identify and trace attack paths.

VII. Related Work

Contemporary microservice anomaly detection strategies fall into three primary categories.

Unsupervised approaches [20], [21], [22], [52] analyze system call patterns through frequency thresholds and sequential context modeling. Early implementations focused on call frequency metrics but evolved to incorporate graphical representations of call sequences and deep learning architectures, enhancing contextual awareness. However, these methods remain vulnerable to "normal drift" as system behaviors gradually diverge from initial training baselines. Existing **Federated Learning-based Anomaly Detection** methods (e.g., MT-FL [13], FL-IDS [53]) focus on detecting anomalies within microservices. They employ techniques like distributed training combined with multi-task learning and feature fusion to identify internal anomalies such as network latency and packet loss. While they excel in privacy preservation and distributed collaboration, they struggle to characterize cross-service causal dependencies. Consequently, they are ineffective against external attackers exploiting internal vulnerabilities to launch cross-service attacks and information leakage attacks.

Untrained detection methods circumvent model training through comparative analysis of service replica behaviors [23], [54], [55]. By tracking ten operational features related to system calls, files, and processes, these techniques calculate Jaccard similarity metrics across time intervals to identify deviations. While eliminating training overhead, they suffer from limited feature correlation analysis and dependence on manual feature engineering.

Rule-based systems employ predefined security patterns for network activity, file access, and system calls [56], [57], [58]. Though initially effective, their static nature conflicts with microservices' dynamic architecture, requiring labor-intensive rule updates and generating excessive false positives that overwhelm security teams. Each paradigm presents distinct trade-offs between detection granularity, adaptability, and operational sustainability. Tracee [59] leans towards low-overhead collection of system calls and event data. While it can be integrated with anomaly detection systems, it lacks intrinsic causal analysis capabilities and cannot directly support attack path tracing.**Deep Learning Tools for Kubernetes** (e.g., KubAnom, DevGraph [60]) leverage the integration of Large Language Models (LLMs) to achieve automated anomaly detection, enhancing capability by incorporating microservice topology graphs. However, these methods depend on massive natural language corpora and significant computational resources, creating technical barriers for small and medium-sized enterprises (SMEs). Furthermore, these deep learning tools do not integrate system-level data such as kernel logs, limiting their scope and making it difficult to cover underlying resource anomalies.The existing microservices anomaly detection systems have been summarized from three major aspects: detection technology, deployment overhead, and forensic capability (Table XVIII).

TABLE XVIII
COMPARISON OF DETECTION METHODS

Detection Method	Detection Type	Multi-Dimensional Data Fusion	Temporal Causality Modeling	Anomaly Forensic Support	Deployment Overhead
STIDE-BoSC [22]	Unsupervised (Sequence Matching)	✗	✗	✗	Low
CHIDS [20]	Unsupervised (Graph + Autoencoder)	✗	✗	✗	Medium
CDL [21]	Unsupervised (Frequency + Autoencoder)	✓	✗	✗	Medium
[52]	Unsupervised (Graph)	✗	✓	✗	High
MT-FL [13]	Unsupervised (Multi-Task Federated Learning)	✓	✗	✗	High
FedTAG [53]	Unsupervised (Federated Learning)	✓	✗	✓	High
ReplicaWatcher [23]	Untrained (Fast Approximation)	✓	✗	✗	Medium
[54]	Untrained (Multi-Criteria Decision Making)	✗	✗	✗	Low
[55]	Untrained (Frequency)	✗	✗	✗	Low
Falco [56]	Rule-based	✓	✗	✗	Low
[57]	Rule-based	✗	✓	✗	Medium
Twisklock [58]	Rule-based	✓	✗	✗	Medium
Trace [59]	Rule-based	✓	✗	✗	Low
DevGraph [60]	Supervised (LLM)	✗	✓	✓	High

VIII. CONCLUSION

This study presents a provenance graph-based detection system addressing microservice anomaly identification through three key innovations: multi-dimensional data fusion, temporal graph networks (TGN) for causality modeling, and an automated forensic analysis module. Experimental evaluations demonstrate superior detection accuracy (15.07% improvement over baselines) across diverse microservice configurations. The integrated forensic component enables full attack chain reconstruction, significantly advancing incident investigation capabilities.

Although effective, three limitations warrant further research: (1) System generalizability across heterogeneous cloud platforms requires enhanced architecture-agnostic modeling; (2) The field urgently needs standardized anomaly datasets with labeled attack patterns; (3) Concept drift mitigation demands self-adaptive models with automated retraining triggers. Future directions should explore federated learning for cross-environment adaptation and blockchain-based dataset sharing frameworks to collectively address these challenges.

REFERENCES

[1] P. Di Francesc, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *J. Syst. Softw.*, vol. 150, pp. 77–97, Apr. 2019.

[2] A. Hannousse and S. Yahiouche, "Securing microservices and microservice architectures: A systematic mapping study," *Comput. Sci. Rev.*, vol. 41, Aug. 2021, Art. no. 100415.

[3] X. Li, Y. Chen, Z. Q. Lin, X. Wang, and J. H. Chen, "Automatic policy generation for inter-service access control of microservices," in *Proc. USENIX Security Symp. USENIX Security Symp.*, Jan . 2021.

[4] S. S. Li et al., "Understanding and addressing quality attributes of microservices architecture: A systematic literature review," *Inf. Softw. Technol.*, vol. 131, Mar. 2021, Art. no. 106449.

[5] Z. Z. Zhang, M. K. Ramanathan, P. Raj, A. Parwal, T. Sherwood, and M. Chabbi, "CRISP: Critical path analysis of large-scale microservice architectures," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2022, pp. 655–672.

[6] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, "Towards resource-efficient reactive and proactive auto-scaling for microservice architectures," *J. Syst. Softw.*, vol. 225, Jul. 2025, Art. no. 112390.

[7] M. S. Haq, T. D. Nguyen, A. S. Tosun, F. Vollmer, T. Korkmaz, and A.-R. Sadeghi, ''SoK: A comprehensive analysis and evaluation of docker container attack and defense mechanisms," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 4573–4590.

[8] Y. He et al, "Cross container attacks: The bewildered eBPF on clouds," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 5971–5988.

[9] V. S. Devi Priya, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Comput. Security*, vol. 135, Dec. 2023, Art. no. 103490.

[10] R. Feng, Z. Yan, S. Peng, and Y. Zhang, "Automated detection of password leakage from public Github repositories," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 175–186.

[11] Q. Y. Zeng, M. Kavousi, Y. H. Luo, L. Jin, and Y. Chen, "Full-stack vulnerability analysis of the cloud-native platform," *Comput. Security*, vol. 129, Jun. 2023, Art. no. 103173.

[12] A. Pereira-Vale, E. B Fernandez, R. Monge, H. Astudillo, and G. Márquez, "Security in microservice-based systems: A multivocal literature review," *Comput. Security*, vol. 103, Apr. 2021, Art. no. 102200.

[13] J. F. Hao, P. Chen, J. Chen, and X. Li, "Multi-task federated learning-based system anomaly detection and multi-classification for microservices architecture," *Future Gener. Comput. Syst.*, 159, pp. 77–90, Oct. 2024.

[14] "Aquasec." Reverse shell. Accessed: Jul. 19, 2023. [Online]. Available: https://kubernetes.io/

[15] O. I. Alqaisi, M. S. Haq, and A. S. Tosun, "Security of containerized computer vision applications," in *Proc. 2nd Int. Conf. Comput. Inf. Technol. (ICCIT)*, 2022, pp. 115–120.

[16] Z. Q. Jian and L. Chen, "A defense method against docker escape attack," in *Proc. Int. Conf. Cryptogr. Security Privacy*, 2017, pp. 142–146.

[17] "Secure Flag." Privilege escalation. Accessed: Jul. 19, 2023. [Online]. Available: https://www.aquasec.com/cloud-native-academy/cloud-attacks/ reverse-shell-attack/

[18] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, "Houdini's escape: Breaking the resource rein of Linux control groups," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2019, pp. 1073–1086.

[19] Y. Q. Sun, D. Safford, M. Zohar, D. Pendarakis, Z. Gu, and T. Jaeger, "Security namespace: Making Linux security frameworks available to containers," in *Proc. 27th USENIX Security Symp. (USENIX Security)*, 2018, pp. 1423–1439.

[20] Y. H. Lin, O. Tunde-Onadele, and X. H. Gu, "CDL: Classified distributed learning for detecting security attacks in containerized applications," in *Proc. 36th Annu. Comput. Security Appl. Conf.*, 2020, pp. 179–188.

[21] A. S. Abed, C. Clancy, and D. S. Levy, "Intrusion detection system for applications using Linux containers," in *Proc. 11th Int. Workshop Security Trust Manag.*, 2015, pp. 123–135.

[22] A. El Khairi, M. Caselli, C. Knierim, A. Peter, and A. Continella, "Contextualizing system calls in containers for anomaly-based intrusion detection," in *Proc. Cloud Comput. Security Workshop*, 2022, pp. 9–21.
[23] A. El Khairi, M. Caselli, A. Peter, and A. Continella, "Replicawatcher: Training-less anomaly detection in containerized microservices," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2024, pp. 1–17.
[24] Z. Y. Li, Q. A. Chen, R. Q. Yang, Y. Chen, and W. Ruan, "Threat detection and investigation with system-level provenance graphs: A survey," *Comput. Security*, vol. 106, Jul. 2021, Art. no. 102282.
[25] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "Flash: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 3552–3570.
[26] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proc. IEEE Symp. Security Privacy (SP)*, 2020, pp. 1172–1189.
[27] Z. J. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 3533–3551.
[28] X. T. Chen, H. Irshad, Y. Chen, and V. Yegneswaran, "{*CLARION*}: Sound and clear provenance tracking for microservice deployments," in *Proc. 30th USENIX Security Symp. (USENIX Security)*, 2021, pp. 3989–4006.
[29] J. Zengy et al., "ShadeWatcher: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Security Privacy (SP)*, 2022, pp. 489–506.
[30] S. Wang et al., "ThreatRace: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3972–3987, 2022.
[31] S. Kasinathan, "You're one misconfiguration away from a cloud-based data breach," 2020. [Online]. Available: https://community.spiceworks.com/t/youre-one-misconfiguration-away-from-a-cloud-based-data-breach/760522/2
[32] C. Warrender, S. Forrest, and B. Pearlmutter, "Detecting intrusions using system calls: Alternative data models," in *Proc. IEEE Symp. Security Privacy*, 1999, pp. 133–145.
[33] Y. Zhang, R. Jin, and Z.-H. Zhou, "Understanding bag-of-words model: A statistical framework," *Int. J. Mach. Learn. Cybern.*, vol. 1, pp. 43–52, Aug. 2010.
[34] "Kubernetes." May 24, 2019. [Online]. Available: https://kubernetes.io/
[35] A. K. Jain et al., "A content and URL analysis-based efficient approach to detect smishing SMS in intelligent systems," *Int. J. Intell. Syst.*, vol. 37, no. 12, pp. 11117–11141, 2022.
[36] A. Ibrahim, A. Bozhinoski, and S. Pretschner, "Attack graph generation for microservice architecture," in *Proc. 34th ACM/SIGAPP Symp. Appl. Comput.*, 2019, pp. 1235–1242.
[37] J. von Kistowski, S. Eismann, N. Schmitt, A. Bauer, J. Grohmann, and S. Kounev, "TeaStore: A micro-service reference application," in *Proc. IEEE 4th Int. Workshops Found. Appl. Self* Syst. (FAS* W)*, 2019, pp. 263–264.
[38] "Sock shop: A microservice demo application." Weaveworks. Accessed: 2017. [Online]. Available: https://github.com/microservices-demo/microservices-demo
[39] "Tetragon—eBPF-based security observability and runtime enforcement," Cilium, Accessed: Jul. 34, 2023. [Online]. Available: https://github.com/cilium/tetragon
[40] J. Flora and N. Antunes, "Evaluating intrusion detection for microservice applications: Benchmark, dataset, and case studies," *J. Syst. Softw.*, vol. 216, Oct. 2024, Art. no. 112142.
[41] L. Yang et al., "{*CADE*}: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Security Symp. (USENIX Security)*, 2021, pp. 2327–2344.
[42] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
[43] S. S. Bokade and P. Kulkarni, "A tool for analyzing and detecting anomalies in unstructured log data," in *Proc. 8th Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solut. (CSITSS)*, 2024, pp. 1–6.
[44] X. Y. Li, H. Y. Zhang, V. H. Le, and P. F. Chen, "LogShrink: Effective log compression by leveraging commonality and variability of log data," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–12.
[45] T. Zhu et al., "General, efficient, and real-time data compaction strategy for apt forensic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3312–3325, 2021.
[46] R. Ahmad, I. Alsmadi, W. Alhamdani, and L. Tawalbeh, "Zero-day attack detection: A systematic literature review," *Artif. Intell. Rev.*, vol. 56, no. 10, pp. 10733–10811, 2023.
[47] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Comput. Commun.*, vol. 198, pp. 175–185, Jan. 2023.
[48] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Inc., Mountain View, CA, USA, Google Technical Report dapper-2010-1, 2010. [Online]. Available: https://static.googleusercontent.com/media/research.google.com/en//archive/papers/dapper-2010-1.pdf
[49] Y. Zuo, Y. Wu, G. Min, C. Huang, and K. Pei, "An intelligent anomaly detection scheme for micro-services architectures with temporal and spatial data analysis," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 548–561, Jun. 2020.
[50] F. Al-Doghman, N. Moustafa, I. Khalil, N. Sohrabi, Z. Tari, and A. Y. Zomaya, "AI-enabled secure microservices in edge computing: Opportunities and challenges," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1485–1504, Mar./Apr. 2023.
[51] J. Chen et al., "TraceGra: A trace-based anomaly detection for microservice using graph deep learning," *Comput. Commun.*, vol. 204, pp. 109–117, Apr. 2023.
[52] S. Jacob, Y. S. Qiao, Y. H. Ye, and B. Lee, "Anomalous distributed traffic: Detecting cyber security attacks amongst microservices using graph convolutional networks," *Comput. Security*, vol. 118, Jul. 2022, Art. no. 102728.
[53] M. Chen, Z. Dai, Y. Li, and Z. Lu, "FedTag: Towards automated attack investigation using federated learning," in *Int. Artif. Intell. Conf.*, 2023, pp. 112–126.
[54] J. Castro, N. Laranjeiro, and M. Vieira, "Exploring logic scoring of preference for DoS attack detection in microservice applications," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2023, pp. 573–584.
[55] U. Zdun et al., "Detection strategies for microservice security tactics," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1257–1273, May/Jun. 2024.
[56] "Falco: Container native runtime security." Accessed: 2022. [Online]. Available: https://falco.org/
[57] X. Gu, Q. Wang, J. Liu, and J. Wei, "Grunt attack: Exploiting execution dependencies in microservices," in *Proc. 54th Annual IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2024, pp. 115–128.
[58] "Twisklock: The most complete container cybersecurity platform," Accessed: 2018. [Online]. Available: https://www.twistlock.com/
[59] "Tracee: Linux runtime security and forensics using EBPF," Accessed: 2023. [Online]. Available: https://aquasecurity.github.io/tracee/latest/
[60] "DevGraph," Accessed: 2022. [Online]. Available: https://www.devgraph.com/

Yanshang Yin received the M.S. degree from Zhejiang Gongshang University, Hangzhou, China, in 2023. She is currently pursuing the Ph.D. degree in computer science with Zhejiang University of Technology, Hangzhou. Her research interests include system security and microservice security.

Tiantian Zhu (Member, IEEE) received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an Associate Professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou. His research interests include mobile security, system security, and artificial intelligence.

Tieming Chen received the Ph.D. degree in computer software and theory from Beihang University, Beijing, China, in 2011. He is currently a Professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China. His research interests include cyberspace security and intelligence security.

Mingqi Lv received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an Associate Professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou. His research interests include system security and artificial intelligence.